The cmlpipe data layer

Technical report on the OpenMRG processing pipeline

Yonatan Haber and Yehonatan Tsitseluk
CellEnMon lab, final-year electrical engineering project
Data-driven discovery of rain-induced and wet-antenna attenuation laws in commercial microwave links

7 October 2026

Abstract

This report documents `cmlpipe`, the data layer of a project on the data-driven discovery of rain-induced and wet-antenna attenuation laws in commercial microwave links (CML). The pipeline converts the raw OpenMRG dataset [1] (364 links, 10 municipal rain gauges, one SMHI gauge, weather-radar and weather data for Gothenburg, June to August 2015) into a verified, read-only and versioned release, `openmrg_v1`. It consists of 23 stages that standardise the raw files without changing any value, attach quality-control flags, compute the path geometry, aggregate all sources to common time bins, define 89 rain events and 62 dry periods from radar and gauges only, freeze a train, validation and test split, and assess the quality of every link. Each output records the hash of the settings that produced it and the code version, so that every file can be traced and unchanged stages are skipped. The products were verified against the values published for OpenMRG (39 of 39 acceptance checks passed), by independent recomputations and by an automated test suite (128 tests passed on 7 October 2026). The release is read through one Python class, `CMLData`, and an interactive explorer. Baseline estimation, attenuation modelling and rain retrieval are outside the scope of the data layer and have not been implemented.

Contents

1 Scope and sources **4**

1.1 Purpose of the data layer 4

1.2 Coverage of this report 5

1.3 Sources of the information 5

1.4 Method and provenance 5

1.5 Conventions of this report 6

2 Overview of the pipeline **6**

2.1 Data levels 6

2.2 Stages 6

2.3 Folder layout 6

2.4 Command line 8
2.5 Mechanisms shared by all stages 9

3 Data and conventions 10
3.1 The OpenMRG dataset 10
3.2 Link data 11
3.3 Gauge and weather data 11
3.4 Radar data 12
3.5 Torslanda case study 12
3.6 Time convention 13
3.7 Units, keys and missing values 14
3.8 Differences between the dataset summary and the data 14

4 Methods 14
4.1 Ingest of the link data 15
4.2 Ingest of gauges, weather and radar 17
4.3 Quality control 18
4.4 Geometry 23
4.5 Aggregation of the link data 27
4.6 Rain from gauges and radar 28
4.7 Rain events and dry periods 31
4.8 Train, validation and test split 35
4.9 Event pairs 36
4.10 Significance and wet runs 38
4.11 Inventory and quality status 39
4.12 Per-link reference state 41
4.13 Figures and the run report 42
4.14 Release 43

5 Verification 45
5.1 Approach 45
5.2 Acceptance of the reference data 45
5.3 Automated tests 47

6 Software organisation **47**

6.1 Structure 47

6.2 Boundary between data layer and research 50

6.3 Generated documentation 50

6.4 Reorganisation 50

7 Access to the data **51**

7.1 Release contents 51

7.2 Programming interface 52

7.3 Explorer 53

7.4 Installation 54

8 Limitations and open work **55**

8.1 Limitations of the data 55

8.2 Open work 56

9 Chronology **56**

References **57**

A Glossary **59**

1 Scope and sources

1.1 Purpose of the data layer

The project studies how rain attenuates the signal of commercial microwave links, the radio links that connect the masts of a cellular network. Rain along the path of a link weakens the received signal, so a network of links can serve as an opportunistic rain sensor. The measured attenuation of a link is written as

$$A(t) = \mathrm{TSL}(t) - \mathrm{RSL}(t) - \mathrm{baseline} = A_{\mathrm{rain}}(t) + \delta(t), \tag{1}$$

where TSL and RSL are the transmitted and received signal levels, the baseline is the attenuation in dry weather, A_{rain} is the attenuation caused by rain along the path, and $\delta(t)$ collects all other contributions, in particular wet-antenna attenuation (water on the antenna covers) and the residual of the baseline. Every segment dx of the path attenuates the signal by $a\,R(x,t)^b\,dx$, so that

$$A_{\mathrm{rain}}(t) = \int_0^L a\,R(x,t)^b\,dx \;\approx\; a\,\bar{R}(t)^b\,L, \tag{2}$$

where $R(x,t)$ is the local rain rate, $\bar{R}$ its mean along the path, L the path length and a, b coefficients that depend on frequency and polarisation. The approximation is exact only for uniform rain. The project generalises it to $A_{\mathrm{rain}} = a\,\bar{R}^b f(L, \bar{R}, Z)$, where f is a correction function to be learned and Z collects properties of the path and the link; uniform rain gives $f = L$. Figure 1 shows the path integral.

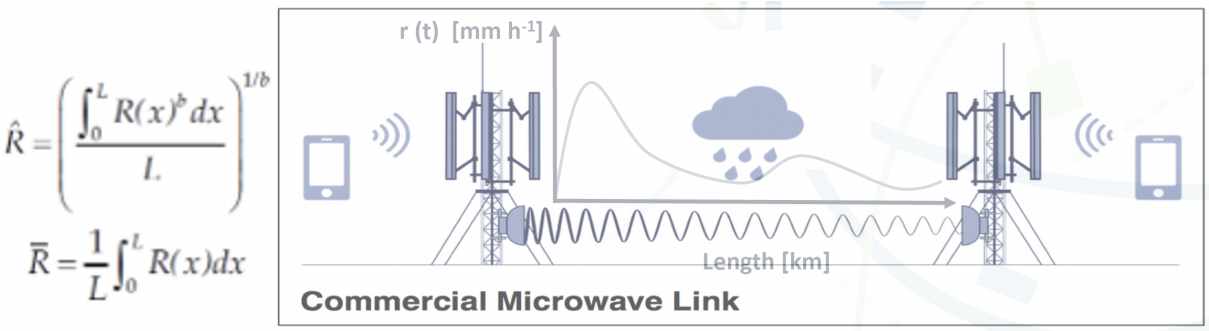


Figure 1: The path integral of a commercial microwave link. Rain varies along the path and is not known. A single link returns $\hat{R}$ (top left), which approximates the path-averaged rain rate $\bar{R}$ (bottom left). Figure from the project proposal; its original source is not identified in the proposal (unverified).

The data layer computes none of the quantities in Eqs. (1) and (2). It provides the measurements (TSL, RSL and their difference for every link), the rain references (gauges and radar, in particular the radar estimate of $\bar{R}$ along each path), the quality information, and the selection of rain events, dry periods and training and test sets from which those quantities will be estimated. All of it is delivered as one verified, frozen dataset, so that every later result can be traced to the same data.

1.2 Coverage of this report

This report covers the data layer of the project, the folder `pipeline/` of the repository, as it stood on 7 October 2026: every stage, assumption, threshold and verification of that folder. Each fact is stated once and referred to afterwards. The research code (`research/`) is not covered; at the time of writing it contains no implementation.

1.3 Sources of the information

The dates are taken from the git history of the repository (commit time stamps) and from the run log of the pipeline, `data/work/openmrg/run_manifest.json` (57 runs between 30 September and 4 October 2026, recorded in UTC). All times in this report are Israel time, UTC+3, unless stated otherwise; times in the data are UTC. The numbers are taken from the documents listed in Table 1, from `pipeline/docs/methods.md`, from the release metadata and from the code. The documents of Table 1 were written during development and are kept locally in `_archive/docs/` (outside version control).

Table 1: Development documents used as sources (under `_archive/docs/`).

Document	Content
`data_notes/openmrg_cml_inspection.md`	Measured facts about the raw CML file and its metadata
`data_notes/openmrg_reference_inspection.md`	Measured facts about gauges, weather, radar and the Torslanda folder
`acceptance/step2_acceptance.md`	Reference data against the published OpenMRG values
`data_notes/openmrg_qc_calibration.md`	Calibration of the quality-control thresholds
`acceptance/step3_pilot.md`	Quality-control pilot on five days
`acceptance/step4_geometry.md`	Acceptance of the geometry stage
`acceptance/step5_aggregation.md`	Acceptance of the aggregation stages
`data_notes/openmrg_event_calibration.md`	Calibration of the event definition
`acceptance/step6_events.md`	Acceptance of events, dry periods and split
`data_notes/openmrg_inventory.md`	Inventory, TSL behaviour and quality status
`acceptance/step7_inventory.md`	Acceptance of significance, inventory and reports

1.4 Method and provenance

The data layer was developed in eight steps (Section 9). Each step followed the same order: the raw data were first inspected and the measured facts written down; thresholds were then calibrated on the full period; a pilot on a few days was inspected visually; and an acceptance report compared the outputs with independent values, using tolerances fixed before the run (Section 5). No code was changed to match a reference number. The code was written by the authors with the assistance of Claude Code; every commit records this in a co-author line.

1.5 Conventions of this report

Identifiers that appear literally in the code or on disk (files, functions, stages, flags, configuration keys, commands) are set in monospace. Paths are relative to `pipeline/` unless they start with `data/`, `_archive/` or name another top-level folder. Configuration keys are relative to `configs/openmrg.yaml`. Appendix A indexes the defined terms.

2 Overview of the pipeline

2.1 Data levels

The pipeline transforms the data in four *levels* (Table 2). Each level is computed from the previous one and never modifies it, so that every step can be repeated and checked independently. The raw files are never written to.

Table 2: Data levels of the pipeline.

Level	Content	Location
Raw	The OpenMRG v1.1 download, never modified	`data/raw/openmrg/`
L1	"Standard": raw values in one common layout (Section 4.1)	`data/work/openmrg/L1_standard/`
L2	Quality-control flags, one bitmask per 10 s sample (Section 4.3)	`data/work/openmrg/L2_qc/`
L3	Statistics per time bin, geometry, events and per-link state (Sections 4.4 to 4.12)	`data/work/openmrg/L3_products/`
Release	All levels and tables in Zarr and Parquet format, read-only (Section 4.14)	`data/release/openmrg_v1/`

2.2 Stages

The processing is divided into 23 *stages*. A stage is one named processing step that reads files, performs one task and writes files. Three stages (`ingest_cml`, `qc_flags`, `aggregate_cml`) work day by day and are called *day stages*. Table 3 groups the stages in pipeline order, and Figure 2 shows what each group does to the data. The outputs of each stage are given in the section that describes it.

2.3 Folder layout

The repository has three parts: `pipeline/` (the data layer), `research/` (the research code) and `docs/` (project documents such as the proposal). The data are not under version control and sit in `data/` at the repository root. Configuration paths of the form `data/...` are resolved against the repository root (`REPO_ROOT` in `src/cmlpipe/config.py`); code, configuration and documentation paths against `PIPELINE_ROOT`, the folder `pipeline/`. Table 4 lists the folders of the work area, with their configuration keys under `paths`.

Table 3: The 23 stages in pipeline order.

Group	Stages	Section
Ingest	`ingest_cml`, `ingest_gauges`, `ingest_met`, `ingest_radar`	4.1, 4.2
Quality control	`qc_stats`, `qc_flags`, `qc_status`	4.3
Geometry	`geometry`	4.4
Aggregation	`aggregate_gauges`, `aggregate_radar`, `aggregate_cml`	4.5, 4.6
Events and split	`events`, `split`, `event_links`	4.7 to 4.9
Inventory	`significance`, `inventory`, `event_links_extra`, `link_state`	4.9 to 4.12
Reports	`inventory_figures`, `quicklooks`, `link_cards`, `run_report`	4.13
Release	`release`	4.14

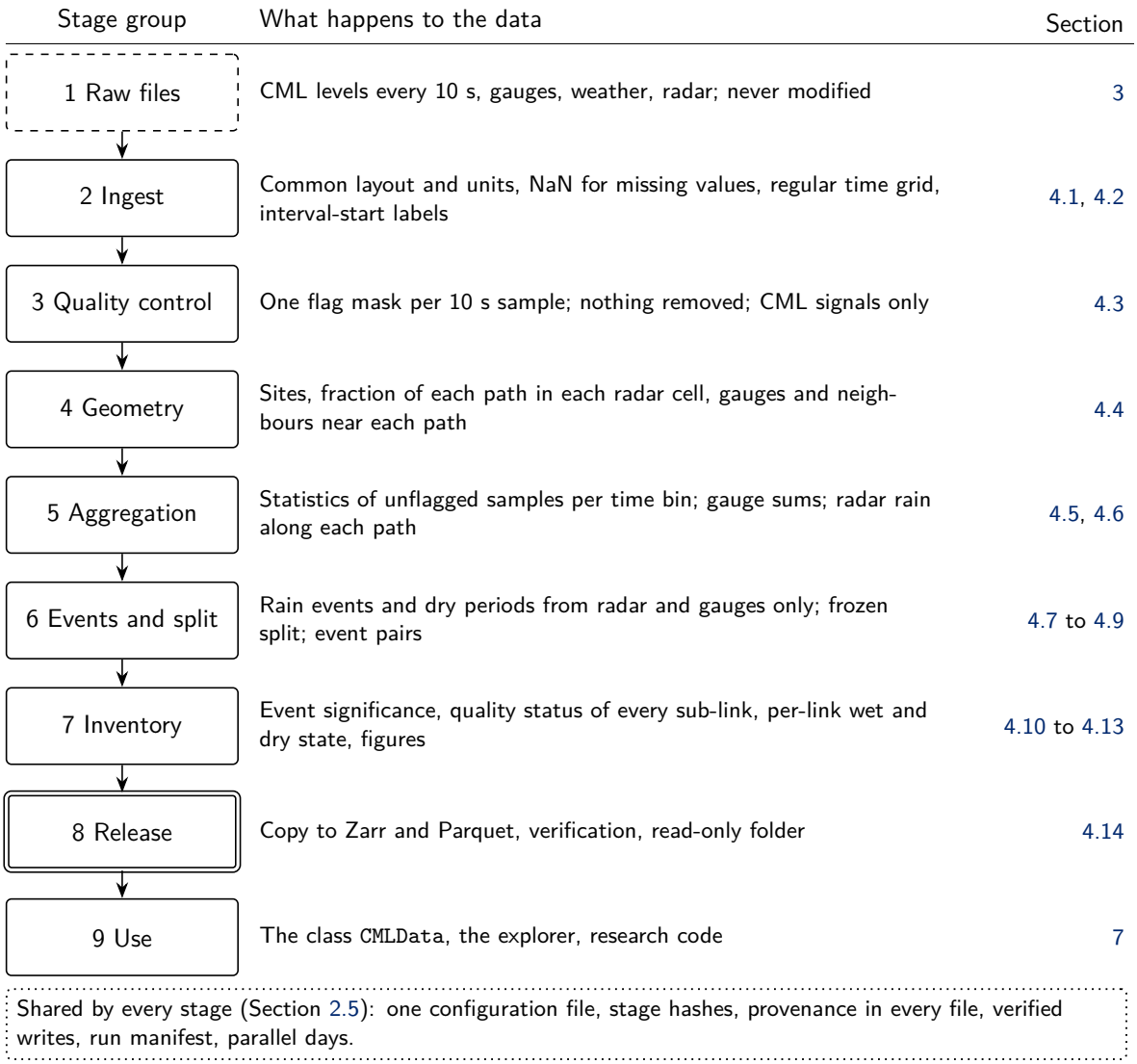


Figure 2: Block diagram of the pipeline, read from top to bottom. Dashed frame: raw input; double frame: frozen release; dotted frame: mechanisms shared by all stages.

Table 4: Folders of the work area `data/work/openmrg/` (keys under `paths`).

Key	Folder	Content
`l1_cml_dir`	`L1_standard/cml`	Daily CML files
`l1_gauges_dir`	`L1_standard/gauges`	City and SMHI gauge files
`l1_met_dir`	`L1_standard/met`	Hourly weather file
`l1_radar_dir`	`L1_standard/radar`	Monthly radar files
`l2_qc_dir`	`L2_qc`	Daily flag files
`l3_cml_dir`	`L3_products/cml`	Daily CML statistics per resolution
`l3_cml_consolidated`	`L3_products/cml/consolidated`	Consolidated core files
`l3_gauges_dir`, `l3_radar_dir`	`L3_products/gauges`, `L3_products/radar`	Gauge and radar statistics
`l3_geometry_dir`	`L3_products/geometry`	Path weights
`l3_events_dir`	`L3_products/events`	Network and per-link reference series
`tables_dir`, `inventory_dir`	`tables`, `tables/inventory`	CSV tables with sidecars
`reports_dir`	`reports`	Figures and the run report
`logs_dir`, `manifest`	`logs`, `run_manifest.json`	Run logs and the run record

2.4 Command line

All stages are started from one command-line interface, `src/cmlpipe/cli.py`, called as `python -m cmlpipe` or through the installed command `cmlpipe`. Table 5 lists its commands and options. A run of `run all` executes every stage in pipeline order and implies `--all` for the day stages; stages whose outputs are up to date are skipped. The earlier form `python pipeline/run.py -config ... -steps a,b` was kept as a thin compatibility layer until 4 October 2026 and then removed.

Table 5: Commands and options of the command-line interface.

Command or option	Effect
`run <stage>[,<stage>...]`	Runs the named stages in pipeline order
`run all`	Runs all 23 stages
`--days D1,D2` or `--all`	Days processed by the day stages
`--force`	Rebuilds outputs even when they are up to date
`--force-split`	Redraws the frozen split; never implied by `--force`
`--workers N`	Runs the day stages on N processes
`--config PATH`	Uses another configuration file
`--version v2`	Builds a new release version next to the existing one
`stages`	Lists the stages
`explore [-release PATH]`	Starts the explorer (Section 7.3)
`guide [-release PATH]`	Rewrites the text files of a release (Section 4.14)

2.5 Mechanisms shared by all stages

2.5.1 Configuration and stage hash

Every path and threshold is stored in one file, `configs/openmrg.yaml` (about 450 lines, each setting with a one-line rationale), read by `load_config` into a `Config` object (`src/cmlpipe/config.py`). Each stage declares the configuration keys it depends on in `STAGE_CONFIG_KEYS`; a key that does not exist raises an error immediately, so a misspelt dependency cannot pass unnoticed. The *stage hash* (`Config.stage_hash`) is the sha256 digest of the canonical JSON of only those keys. Editing the radar settings therefore changes the hash of the radar stages but not of the CML stages. The hash of the whole configuration (`config_hash`) is also computed and stored for information. Dataset paths and output folders are not part of any hash, so moving the data folder does not invalidate the outputs.

A stage is skipped when every output carries the current stage hash and its inputs are unchanged. Inputs are compared through the stage hashes of the input files or through an *input signature*, a sha256 digest over everything the stage reads. The option `--force` rebuilds the outputs.

2.5.2 Provenance

Every output file stores, as global attributes, the stage name (`stage`), the stage hash (`stage_hash`), the configuration hash (`config_hash`), the git commit of the code (`code_version`, with the suffix `+dirty` when the working tree has uncommitted changes) and the creation time in UTC (`created_utc`); the function is `provenance_attrs` in `src/cmlpipe/provenance.py`. Files that cannot hold attributes, such as CSV tables, receive a sidecar file `<name>.meta.json` with the same information (`write_sidecar`). Each run appends one record to `run_manifest.json` (`append_to_manifest`) with the command, the code version, the stages, per-day records and the run time of each stage, and writes a log file `logs/<run_id>.log`, where the run identifier is the UTC start time, for example `20260930T141502Z` (`src/cmlpipe/logging_utils.py`).

2.5.3 Verified writes

Every file is first written under a temporary name (`<name>.tmp`), then re-opened, compared with the data in memory and only then renamed to its final name. An interrupted run therefore never leaves a partial file, and an encoding error is detected at the moment of writing.

2.5.4 Parallel days

The day stages run on several processes with `--workers N` (`src/cmlpipe/parallel.py`, function `run_day_task`). On Windows new processes are started with the method “spawn”, so every task is a module-level function that receives the configuration as a dictionary and rebuilds it (`config_from_parts`). An exception in one day is turned into a “failed” record (`_guarded`), so

that the records of the other days are not lost. The records are returned and written to the manifest in day order. A test confirms that one and four workers produce identical files.

2.5.5 File access

Large NetCDF files are read with the `netCDF4` library and sliced before loading; `xarray` is used where its convenience does not cost time. Many NetCDF files are never opened at once with `xarray`, because that is slow on Windows, and the parallel computing library `dask` is not used. Where a later stage needs a few variables of all days, a consolidated core file is built once (Section 4.7). The raw CML file is never loaded whole (Section 3.2).

3 Data and conventions

3.1 The OpenMRG dataset

The input is the OpenMRG dataset, version 1.1 [1, 2], collected by the Swedish Meteorological and Hydrological Institute (SMHI), Ericsson and the operator Hi3G in Gothenburg, Sweden, from 1 June to 31 August 2015. This period is called *JJA* (June, July, August) below. The dataset is published under the licence CC BY-SA 4.0 and is about 318 MB as a compressed download. Table 6 lists its parts; the following subsections give the facts that were measured on the files before any processing.

Table 6: Raw input files of OpenMRG (under `data/raw/openmrg/`).

File	Step	Content
`cml/cml.nc`	10 s	TSL and RSL of all sub-links (Section 3.2)
`cml/cml_metadata.csv`	n/a	One row per sub-link: link, direction, end coordinates, frequency, length, polarisation, antenna diameters
`gauges/city/CityGauges-2015JJA.csv`	1 min	10 municipal (City) gauges, rain amount in mm per minute
`gauges/city/CityGauges-metadata.csv`	n/a	Name, location, coordinates, type and resolution of the City gauges
`gauges/smhi/GbgA-71420-2015JJA.csv`	15 min, 1 h	SMHI gauge GbgA (rain amount per 15 min) and hourly weather
`gauges/smhi/GbgA-71420-metadata.csv`	n/a	Station and parameter description
`radar/radar.nc`	5 min	NORDRAD composite of reflectivity (Section 3.4)
`Torslanda-event-2015-07-28/`	n/a	Case-study event (Section 3.5)
`readme.txt`	n/a	Dataset description, including the meaning of every time stamp

3.2 Link data

A *link* connects two *sites* (masts) and carries two directions, each called a *sub-link*. OpenMRG has 364 links and 728 sub-links on 363 hops between 418 sites, measured by Ericsson MINI-LINK radios with automatic transmit power control (ATPC) documented as disabled [1]. Frequencies range from 7.456 to 38.682 GHz and lengths from 0.142 to 15.228 km; higher frequencies are used on shorter links. The *total loss* of a sub-link is TL = TSL − RSL in dB; rain increases TL.

The raw file `cml.nc` (4.6 GB) is a NetCDF-4 file in which the variables `rsl` and `tsl` (float32, dBm) have the dimensions (time, sublink) and are stored contiguously, without chunks or compression. Reading one day therefore costs 8640 × 728 × 4 B, about 25 MB per variable, in one contiguous block; for this reason the file is never loaded whole but read one day at a time. Missing values are coded as 10^{10} (both `_FillValue` and `missing_value`); no other missing-value form was found. The time axis runs from 31 May 2015 23:59:00 to 1 September 2015 00:00:00 UTC in 794 887 steps of exactly 10 s, without duplicates or gaps, so the file contains six steps before 1 June and one step on 1 September. According to `readme.txt` a time stamp marks the start of the sampling window, and "near" and "far" denote the transmitting and the receiving node of each sub-link.

The metadata file has one row per sub-link, with exactly one direction A and one direction B per link; the sub-link numbers of the metadata and of `cml.nc` are the same set (1 to 728). Length and polarisation are identical for both directions of every link. Near and far are swapped between A and B in all links, which confirms that near and far are defined per sub-link (transmitter and receiver) and not per link. Antenna diameters swap in the same way; 36 links have different diameters at their two ends. The frequency difference between the two directions has a median of 1.008 GHz and a maximum of 1.498 GHz; direction A has the higher frequency in 148 links and the lower in 215, and one link (10303, 28.0 GHz, 4.6 km) uses the same frequency in both directions. Polarisation is spelled `Vertical` in 722 rows and `Horizontal` in 6 rows (links 10067, 10203 and 10283).

Every finite value is an exact multiple of 0.1 dB. TSL changes in whole dB (steps of 1 dB and, rarely, 2 dB). RSL lies on a grid of 1/3 dB that is stored rounded to 0.1 dB, so its steps are 0.3 dB and 0.4 dB (three steps make 1 dB). The levels are therefore *quantised*, which matters for every later noise model. The published extremes are RSL from −93.7 to −25.3 dBm and TSL from −10 to 26 dBm; below about −90 dBm RSL contains only receiver noise [1].

3.3 Gauge and weather data

The City file has the column `Time_UTC` and one column per gauge (Jarn, Torp, Bergsj, Torsl, Chalm, Tole, Barl, Drakeg, Lbom, Askim), with 132 480 rows (92 days of 1440 minutes) at exactly 60 s, without duplicates or gaps. Missing values are written as the text `NA` and occur only at the gauge Barl, in one block of 2880 rows (2 days at the start of June). No value is negative. Seven gauges are weighing gauges with a resolution of 0.1 mm and three (Drakeg, Lbom, Askim) are tipping-bucket gauges with a resolution of 0.2 mm. The height of the City

gauges is not documented; the dataset paper gives about 1.5 m as typical [1], and this value is stored as an assumption. The metadata file is encoded in UTF-8 with a byte-order mark.

The SMHI file of station GbgA (Göteborg A, station 71420) has 8832 rows (92 days of 96 quarter hours). The precipitation column `Pvol_mm` has no missing value, a resolution of 0.1 mm and a quality letter per row (G, quality controlled and approved, 8828 rows; Y, uncontrolled or suspicious, 4 rows). The gauge is a weighing gauge 2 m above ground. The weather variables (temperature `Ta_C`, humidity `RH_%`, sea-level pressure `Pmsl_hPa`, mean wind speed `Vmean_m/s`, wind direction `Vdir_deg`, gust `Vg_m/s`) are filled only on the full hour; their valid counts are given in Table 39. The metadata file is encoded in Latin-1 and consists of two free-text blocks rather than one table, so it is parsed by a dedicated reader (`read_smhi_metadata`).

3.4 Radar data

The radar file contains a subset of the Swedish NORDRAD composite at the lowest elevation (0.5°) on 37 × 48 cells of 2 km, covering the network and about 10 km around it [1]. The composite is adjusted to gauges and is therefore not independent of them [1]. The variable `data` has the dimensions (time, y, x) and is stored as 32-bit integers from 0 to 255, with the value 255 marking a missing cell. The file attributes define the decoding to reflectivity (Section 4.6), which `netCDF4` and `xarray` apply automatically when the file is opened, so it must not be applied a second time, and the Z-R relation, stored as text. The most frequent raw value is 0 (−30 dBZ), and the largest valid raw value is 222. Of all cell values, 439 134 are missing, so 99.067 % are valid; 379 time steps have at least one missing cell. The time axis has 5 min steps from 1 June 2015 00:05 to 1 September 2015 00:00 UTC, without duplicates or gaps; its length, the steps without any valid cell and the largest reflectivity are compared with the paper in Table 39.

The coordinates `x` and `y` are cell centres in metres of a polar stereographic projection (`+proj=stere +lat_ts=60 +ellps=bessel +lon_0=14 +lat_0=90`); `y` decreases, so the first row is the northern edge. The file also holds two-dimensional latitude and longitude of the cell centres, which agree with the inverse projection of `x` and `y` to within 0.2 m. The projection string contains no datum shift, so it is not stated whether the coordinates refer to WGS84; they are treated as WGS84.

3.5 Torslanda case study

The folder `Torslanda-event-2015-07-28/` documents a convective event at the City gauge Torsl on 28 July 2015. It contains an interactive page with the data of the dataset paper's case-study figure and a radar animation. The page shows the window 14:30 to 17:30 in raw labels (UTC). In the embedded series the first non-zero gauge value is labelled 14:59 and the last 16:54; the radar trace peaks at 41.6 dBZ at 15:35; and RSL of sub-link 115 (link 10227) reaches its minimum of −73.6 dBm at 15:06. The whole-day total at Torsl is 19.4 mm, of which 0.9 mm fell outside the window. The event total and peak are compared with the paper in Table 39. The folder is used only for verification.

3.6 Time convention

The sources label their values differently (Table 7). The pipeline therefore uses one convention: every value is labelled by the start of the interval it describes, so a value at time t with step Δt covers $[t, t + \Delta t)$ (*interval-start labelling*). Sources labelled at the end of their interval are shifted once, at ingest. Figure 3 shows the shift for the peak minute of the Torslanda event; the dataset paper states that the peak was recorded between 15:04 and 15:05, which confirms the direction of the shift.

Table 7: Meaning of the raw time stamps and the shift applied at ingest.

Source	Meaning of the raw time stamp	Shift (key)
CML	Start of the 10 s sampling window	none
City gauges	End of the minute (the interval starts on second 01 of the preceding minute)	-60 s (`city_gauges.time_shift_s`)
SMHI precipitation	End of the 15 min interval	-900 s (`smhi_gauge.time_shift_s`)
SMHI temperature, humidity, pressure	Instantaneous value	none
SMHI wind speed and direction	Mean of the preceding 10 min	none; meaning stored
SMHI gust	Maximum of the preceding hour	none; meaning stored
Radar	"Timestamp on which the reflectivity was sampled"	none

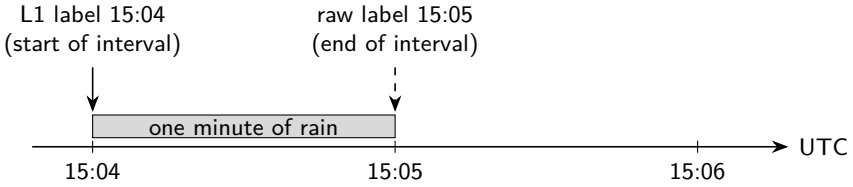


Figure 3: Interval-start labelling: the raw City label marks the end of the minute and is shifted by -60 s.

The weather variables share one time axis but have three different meanings, so no single shift can be correct for all of them; their stamps are kept and each variable stores its meaning as an attribute. The radar documentation does not state whether its stamp is the start or the end of a scan or of a 5 min composite. This was tested empirically (`radar_timing_lag.py`, archived): the 5 min sums of the 10 City gauges were correlated with the radar shifted by k steps of 5 min, using the radar cell nearest to each gauge. The mean correlation was 0.404, 0.471, 0.513, 0.410 and 0.323 for $k = -2, -1, 0, +1, +2$; the peak at $k = 0$ gives no evidence of an end-of-interval label, so no shift is applied and a radar stamp t is assigned to the bin $[t, t + 5\ \text{min})$.

The period is 1 June 2015 00:00 to 1 September 2015 00:00 (keys `period.start`, `period.end`). For interval-start sources it is a half-open window, $[\text{start}, \text{end})$. For instant-labelled sources (radar, weather) the window is closed, because a stamp exactly at 1 September 00:00 is a real measurement at the closing instant (key `period.window_end_inclusive`); their time axes run

from the first to the last raw stamp, so that no step is added. The CML time axis of the release therefore has 794 880 steps (92 days of 8640 steps); the seven raw steps outside JJA (Section 3.2) are not used.

3.7 Units, keys and missing values

Signal levels are in dBm and losses in dB, rain amounts in mm and rain rates in mm/h, frequencies in GHz and link lengths in km in L1. All distances computed by the pipeline are in metres in the Swedish projection SWEREF 99 TM (EPSG:3006). The identifiers are listed in Table 8. A missing value is NaN and means "unknown"; it never means zero.

Table 8: Identifiers.

Identifier	Example	Meaning
`cml_id`	10227	Link, the raw column `Link`
`sublink_id` (L1)	`sublink_1`, `sublink_2`	Sub-link 1 transmits from `site_0` to `site_1` (raw direction A); sub-link 2 the reverse (raw direction B)
`sublink_id` (release)	`10227_1`	`<cml_id>_<1|2>`
`raw_sublink_id`	115	Raw sub-link number, kept for traceability
`gauge_id`	`Torsl`, `GbgA`	Gauge name
`site_id`	0 to 417	Mast
`event_id`	`E20150727_2140`	"E" and the start time of the event
`run_id`	`E20150727_2140_R02`	Wet run within an event

3.8 Differences between the dataset summary and the data

The inspection found several differences between the project's summary of the dataset and the files. The file names the identifier doi:10.5281/zenodo.6673750, while the configuration uses doi:10.5281/zenodo.7107689 for version 1.1; the relation between the two was not verified, and both are stored in every L1 file (`source_doi`, `raw_file_doi`). The RSL resolution is a 1/3 dB grid stored at 0.1 dB rather than a plain 0.3 dB step (Section 3.2). The CML file is not limited to JJA. The radar completeness quoted in the dataset paper is the share of time steps with at least one valid cell (Table 39), not the share of valid cell values (Section 3.4). The radar values are stored as 32-bit integers, not as unsigned bytes. The SMHI wind variables are 10 min means and a 1 h maximum rather than hourly values. The dataset paper states 99.95 % completeness for GbgA in its text and 100 % in a table; the file has no missing precipitation value.

4 Methods

Each subsection describes one group of stages in pipeline order. The text follows the same order in every subsection: purpose and rationale, method, verification, and the issues found during development with their corrections. Thresholds are listed in tables with their configuration key.

4.1 Ingest of the link data

4.1.1 Purpose

The stage `ingest_cml` brings the raw link data into the common L1 layout, so that no later stage needs to know the raw format, and so that the data can be used by the OpenSense tools `pycomlink` and `poligrain`. The L1 layout follows the OpenSense naming conventions [3] with two unit choices of this project: frequency in GHz and length in km (the OpenSense sample files use MHz and m). No measured value is changed.

4.1.2 Method

The stage reads one UTC day [00:00, 24:00) at a time from `cml.nc` (`src/cmlpipe/io/openmrg.py`, functions `_read_raw_day` and `load_cml_day`). The file is opened without automatic masking, and the missing-value code (key `cml.raw_missing_value`) is replaced by NaN explicitly, so that the conversion is visible and counted. The samples are placed on a regular 10 s grid: a missing time stamp becomes a row of NaN, and of a duplicated time stamp the first occurrence is kept while the number of dropped duplicates (and of those with different values) is logged and recorded. The metadata are read by `load_cml_metadata`, which produces one row per link and sub-link and normalises the polarisation spelling to `v` or `h` (key `cml.polarization_map`); an unknown spelling, a duplicated sub-link number or a duplicated link and direction raises an error. Direction A becomes `sublink_1` and direction B `sublink_2` (key `cml.direction_to_sublink`), and site 0 is the transmitter of sub-link 1. A raw sub-link without metadata is dropped; a link with only one direction keeps an empty second sub-link with NaN data.

The L1 dataset has the dimensions (time, `sublink_id`, `cml_id`); the variables `tsl`, `rsl` (dBm) and `tl` = `tsl` − `rsl` (dB); and the coordinates `raw_sublink_id` and `direction` (traceability), `tx_site` and `rx_site`, `frequency` (GHz) and `polarization` per sub-link, and `length` (km), the coordinates of both sites and the antenna diameters per link. The values are stored as 16-bit integers with a scale factor of 0.1 dB, zlib compression level 4 and the fill value −32 768 (keys under `encoding`). Because every raw value is a multiple of 0.1 dB this encoding is lossless; `write_l1_day` writes the file, re-opens it and calls `check_roundtrip`, which fails, and leaves no file, if any value differs by more than 10^{-5} dB (`encoding.roundtrip_atol_db`). The function `write_sublink_id_map` writes `tables/sublink_id_map.csv`, the translation from raw sub-link numbers to the new identifiers. Every day file is validated by `validate_cml_l1` (`src/cmlpipe/io/schema.py`) as read back from disk; Table 9 lists the checks of all L1 validators. A validator separates hard errors from warnings: an implausible but possible value only produces a warning.

For each day the manifest receives statistics computed on the file as re-read from disk (`day_statistics`): number of steps, NaN counts, sub-links without data and value ranges. The stage writes one file per day, `L1_standard/cml/cml_10s_<day>.nc`, 92 in total.

Table 9: Checks of the L1 validators (keys under `validation` and `reference_validation`).

Check	Rule
Structure	Required variables and dimensions in the required order; units present
Time axis	Monotonic, regular, interval start, without time zone
Frequency	1 to 100 GHz (a value in MHz fails)
Length	0.01 to 50 km (a value in metres fails)
Coordinates	Latitude −90 to 90°, longitude −180 to 180°; for gauges and radar 50 to 70° N and 5 to 25° E
Polarisation	`v` or `h`
Total loss	`tl` equals `tsl` − `rsl` within 10^{-3} dB
Plausible levels (warning)	RSL −110 to −10 dBm, TSL −20 to 40 dBm
Plausible rain (warning)	Rain rate up to 300 mm/h, reflectivity −35 to 75 dBZ
Z-R consistency	Stored rain rate equals the Z-R relation within a relative 10^{-5}

4.1.3 Verification

The adapter is tested on a small synthetic raw dataset (`tests/conftest.py`) that copies the real raw layout and contains on purpose a missing-value code, a missing time stamp, a duplicated time stamp with a different second value, time stamps of the previous and next day, a link with only direction A and one with only direction B, polarisation spelled `Vertical`, `V` and `horizontal ` (with a trailing space), a sub-link without metadata, and a link whose direction A has a higher sub-link number than direction B. The tests check the metadata mapping, the normalisation, the transmitter sites, the shape and grid, the placement of values, the conversion to NaN, `tl`, the validator, the lossless round trip, that a value that is not a multiple of 0.1 dB makes the writer fail, and that the command line writes, skips and, with `--force`, rebuilds a day. Further validator tests break a valid dataset on purpose (dimension order, units, non-monotonic or irregular time, MHz, metres, polarisation, `tl`) and confirm that each break is caught, while implausible values only warn. On the real data, tests check the day 2015-07-28: shape and validation, equality of link 10227 with a direct `netCDF4` read of the raw file, and a lossless round trip. An independent script compared the same day with the raw file read by a different route (dimension order, time range, values, NaN positions, `tl`, sub-links without data, metadata mapping, horizontal links, encoding).

4.1.4 Issues found and corrections

The original independent check script was not committed and was lost when the check scripts were moved on 30 September 2026; it was reconstructed from its recorded output, which is stated in `_archive/checks/README.md`.

4.2 Ingest of gauges, weather and radar

4.2.1 Purpose

The stages `ingest_gauges`, `ingest_met` and `ingest_radar` (`src/cmlpipe/io/openmrg_reference.py`, run from `src/cmlpipe/io/ingest.py`) bring the rain and weather references into L1 layouts of their own, with the time convention of Section 3.6.

4.2.2 Method

`load_city_gauges` reads the City file, converts `NA` to NaN (key `city_gauges.na_tokens`), counts missing, duplicated and inserted rows, applies the time shift and places the values on a regular 1 min grid. `load_smhi_gauge` reads the SMHI precipitation, applies its time shift and converts the quality letters to numbers (G = 1, Y = 2, none = 0; key `smhi_gauge.qc_codes`); an unknown letter raises an error. `load_smhi_weather` reads the six weather variables with their quality codes and stores the meaning of each time stamp as an attribute. `load_radar_month` reads one calendar month of the radar, clipped to the closed period window (`month_window`), checks that the reflectivity was decoded exactly once, and computes the rain rate by the Z-R relation of Section 4.6. Station positions are projected to EPSG:3006 by `lonlat_to_metric`.

Each file is written by `write_l1_reference` as float32 with zlib level 4 and the quality codes as unsigned bytes, then re-opened and required to be identical, including the positions of NaN (`check_roundtrip_exact`), and validated by `validate_gauge_l1`, `validate_met_l1` or `validate_radar_l1`. The outputs are `city_1min.nc`, `smhi_15min.nc`, the weather file, the three radar files `radar_5min_2015-06.nc` to `radar_5min_2015-08.nc`, and `tables/gauges.csv` with one row per gauge (network, type, coordinates, resolution, height, interval, completeness, totals). The gauge table is built from the written L1 files, not from the raw files.

4.2.3 Verification

Tests on synthetic files check the City shift and sums, the counting of missing, duplicated and inserted rows, the SMHI shift, sums and quality codes, that the weather is kept as in the file, that the radar is decoded once and converted by the Z-R relation, that every raw stamp of an instant-labelled source inside the closed window appears in L1 and no other, that time zones, wrong units and a wrong radar dimension order are rejected, and that the command line writes, validates and skips the reference files. The L1 reference data were compared with the published values; all 39 checks passed (Table 39).

4.2.4 Issues found and corrections

The first version of the radar and weather ingest used the half-open period window and a full time grid. This dropped the valid radar stamp at 1 September 2015 00:00 and introduced a spurious step without data at 1 June 2015 00:00. The error was found because the acceptance

test compared time labels and not only counts. It was corrected by the closed window for instant-labelled sources (Section 3.6), and the radar and weather files were rebuilt. All L1 files were rebuilt once more after the introduction of per-stage hashes, and their values and coordinates were confirmed to be identical to the previous files for one CML day and every gauge, weather and radar file.

4.3 Quality control

4.3.1 Purpose

Measured CML signals contain missing samples, frozen values, receiver clipping and isolated outliers. If such samples are treated as rain, the attenuation laws estimated later are biased; if they are deleted or repaired, the record of what happened is lost. The quality-control stages therefore follow the rule "flag, do not fix": L1 is never modified, and L2 stores, for every 10 s sample, a 16-bit *flag* bitmask that describes what is suspicious about it. The flags use only the CML signals and their metadata, never radar, gauges or weather, so that the rain reference cannot enter the quality labels.

4.3.2 Flags

Table 10 lists the eight flags, defined once as `QCFlag` in `src/cmlpipe/qc/flags.py` and evaluated by the functions of `src/cmlpipe/qc/rules.py`; their thresholds are listed in Table 12. A *blocking* flag removes the sample from statistics; the flags `tsl_step` and `flatline` only inform. The blocking sets differ by use (Table 11).

Table 10: Quality-control flags.

Bit	Flag	Rule	Function
0	`missing_rsl`	RSL is NaN	`flag_missing`
1	`missing_tsl`	TSL is NaN	`flag_missing`
2	`out_of_range`	RSL or TSL outside its valid range	`flag_out_of_range`
3	`rx_floor`	RSL near the floor level of the sub-link for a minimum duration, on sub-links whose floor is deep below their median and at the receiver limit; the attenuation is then only a lower bound	`flag_rx_floor`
4	`spike`	A short run that deviates strongly from a centred rolling median, with calm neighbours that agree with each other	`flag_spike`
5	`tsl_step`	TSL changes by at least the TSL quantum from the previous valid sample	`flag_tsl_step`
6	`flatline`	RSL exactly constant over a minimum number of valid samples	`flag_flatline`
7	`network_gap`	Sample missing while a minimum share of the alive sub-links is missing at the same step (a collector outage)	`flag_network_gap`

Table 11: Blocking sets.

Use (key)	Blocking flags
L2 summaries (`qc.blocking_flags`)	`missing_rsl`, `missing_tsl`, `out_of_range`, `rx_floor`, `spike`, `network_gap`
RSL statistics (`aggregate.blocking_flags.rsl`)	`missing_rsl`, `out_of_range`, `rx_floor`, `spike`, `network_gap`
TSL statistics (`aggregate.blocking_flags.tsl`)	`missing_tsl`, `out_of_range`, `network_gap`
TL statistics	Union of the RSL and TSL sets

Three terms of Table 10 need definitions. The *floor level* of a sub-link is the lowest occupied RSL level L that holds a minimum number of samples within $[L, L + \text{window}]$ (`floor_level`). The *scaled MAD* is 1.4826 times the median absolute deviation, which equals the standard deviation for Gaussian noise but ignores rare outliers; the spike threshold is the larger of a multiple of the scaled MAD and a minimum absolute deviation. A sub-link is *alive* on a day when a minimum share of its RSL samples is valid on that day (`alive_columns`); only missing samples of alive sub-links can carry `network_gap`, because a sub-link that is missing all day is missing for its own reason. Lengths of runs are counted with `run_lengths` and `run_bounds`.

Table 12: Quality-control parameters (keys under `qc`).

Key	Value	Rationale
`context_hours`	12 h	Runs that cross midnight are judged in full
`stats.alive_min_valid_fraction`	0.5	Definition of an alive sub-link
`stats.floor_min_samples`	30	5 min of samples in total
`stats.floor_window_db`	1.0 dB	Width of the floor level
`flags.valid_range.rsl_dbm`	−100, −20 dBm	Safety net outside the published extremes
`flags.valid_range.tsl_dbm`	−15, 30 dBm	Safety net outside the published extremes
`flags.rx_floor.margin_db`	1.0 dB	About three quantisation steps
`flags.rx_floor.depth_db`	20 dB	Floor well below the median
`flags.rx_floor.receiver_floor_max_dbm`	−87 dBm	Floor at the receiver limit (calibration)
`flags.rx_floor.min_duration`	10 min	A fade that only touches the floor is not clipping
`flags.spike.window`	5 min	Centred rolling median of 31 samples
`flags.spike.n_mad`	6	Robust deviation threshold
`flags.spike.min_abs_db`	5 dB	Calibrated (Table 15)
`flags.spike.max_len`	3	At most 30 s; longer excursions are signal
`flags.spike.neighbour_agree_db`	1.5 dB	The signal returns after the spike
`flags.spike.neighbour_max_dev_db`	1.5 dB	It returns to a calm level
`flags.flatline.min_duration`	12 h	Calibrated; counted in valid samples
`flags.tsl_step.min_db`	1.0 dB	The TSL quantum
`flags.network_gap.fraction`	0.05	Calibrated (Table 16)

4.3.3 Stages

The stage `qc_stats` (`src/cmlpipe/qc/stage.py`, `run_qc_stats`) passes over all 92 days and computes, per sub-link, a histogram of RSL and TSL in 0.1 dB bins (accumulated with `numpy.bincount`), the median, percentiles derived from the histogram, and the floor level. It writes `tables/sublink_period_stats.csv` and `sublink_period_hist.npz`. The stage `qc_flags` then flags each day. `load_with_context` loads the day together with part of each neighbouring day (`context_hours`); `compute_flags` applies the rules, `flag_day` produces the L2 dataset and the daily summary rows, and `write_l2` writes `L2_qc/cml_flags_10s_<day>.nc` (uint16, zlib level 4) by the verified-write procedure. The daily summary `tables/qc_daily_summary.csv` holds, per day and sub-link, the count of every flag; `upsert_daily_summary` replaces only the rows of the days that were processed. The summary also contains the correlation of the two directions of each link after removal of a 10 min rolling median (`pair_correlation`, key `qc.summary.pair_corr_detrend_window`). `network_gaps_table` collects from all L2 files the periods in which the missing share of alive sub-links exceeded the threshold (`tables/network_gaps.csv`). Inside the stage a day is handled as a two-dimensional array with one column per (sub-link, link) pair.

The stage `qc_status` assigns a first status per sub-link (`tables/sublink_qc_status.csv`, `sublink_status`): unusable when the sub-link has no RSL or no TL, an RSL completeness below 50 %, `rx_floor` above 20 % or blocking flags above 20 % of the valid samples, and degraded when blocking flags exceed 5 % (keys under `qc.status`). It refuses to run until every day of the period has been flagged. This status was later replaced by the quality status of Section 4.11; the table is kept for reference.

4.3.4 Calibration of the thresholds

The starting thresholds were checked on all 92 days with the script `qc_calibration.py` (archived) and the outputs of `qc_stats`. Of the 728 sub-links, 720 have RSL at some time and 8 never; 16 never have TL.

The receiver floor was first defined as the 0.1st percentile of RSL with the same depth. Table 13 shows why that definition was rejected: no sub-link has its 0.1st percentile within −91 to −89 dBm, and the difference between the median and that percentile never exceeds 21 dB, so the percentile describes the deepest rain fades and not clipping. 78 sub-links reach a minimum between −91 and −89 dBm, but on most of them the minimum is a single sample (median 1, 95th percentile 3 samples). The longest run within 1 dB of a sub-link's own minimum has a median of 0.3 min, a 95th percentile of 1.3 min, a 99th percentile of 18.1 min and a maximum of 79.2 min; 11 sub-links have such a run of at least 10 min. The floor level falls into two separated groups: 13 sub-links between −87.8 and −93.7 dBm, and all others at −82.1 dBm or higher. Of the 13, 10 lie deep enough below their median to be flagged (links 10046, 10071, 10074, 10140, 10148, 10163, 10256, 10315, 10334 and 10354, all `sublink_1`); the other three (10041, 10134, 10135) are nearly without data (0.05 % complete) and their median is the floor

itself. One sub-link (10082) holds 79 min near −67.4 dBm, far above the receiver limit, which is a signal state and not clipping.

Table 13: RSL statistics over the 720 sub-links with RSL (dBm and dB).

Quantity	p0	p1	p5	p25	p50	p75	p95	p100
RSL minimum	−93.7	−90.0	−90.0	−82.5	−75.8	−66.4	−57.0	−39.8
RSL 0.1st percentile	−92.8	−66.4	−60.8	−55.5	−52.6	−50.4	−45.4	−34.4
Median minus 0.1st percentile	0.0	0.6	4.4	6.3	8.2	10.4	14.1	21.1

The spike threshold was calibrated on the changes of RSL between adjacent valid samples (565 million pairs, Table 14). The most frequent non-zero change is 0.3 dB on 676 sub-links and 0.4 dB on 44; 86.9 % of all non-zero changes are 0.3 or 0.4 dB and 0.049 % are at least 5 dB. The tail hardly depends on link length. The deviation from the 5 min rolling median has a 99th percentile of 1.6 dB, a 99.9th of 3.8 dB and a 99.99th of 6.9 dB on the dry day 2 June 2015, and 0.7, 2.5 and 7.2 dB on the rain day 28 July 2015. Table 15 shows trial counts of the full rule. At 3 to 4 dB the count doubles to quadruples on the dry day, which indicates that the rule reaches ordinary variability; 5 dB, about 12 quantisation steps and just below the 99.99th percentile of the changes, was chosen. On link 10227 during the Torslanda event (14:50 to 17:00), with a maximum deviation of 9.4 dB, no sample was flagged at any of the tested thresholds.

Table 14: Absolute RSL change between adjacent valid 10 s samples (dB) by length class.

Length class	Sub-links	p50	p90	p99	p99.9	p99.99
All	728	0.0	0.3	1.0	2.5	5.4
< 1 km	130	0.0	0.3	0.7	1.9	5.6
1 to 3 km	328	0.0	0.3	0.9	2.8	5.6
> 3 km	270	0.0	0.3	1.0	2.8	5.1

Table 15: Trial of the spike rule: flagged samples (sub-links) by minimum absolute deviation.

Minimum deviation	2015-06-02	2015-07-28
3 dB	1035 (270)	432 (122)
4 dB	535 (154)	178 (75)
5 dB	283 (85)	92 (44)
6 dB	154 (59)	72 (32)

The longest exactly constant RSL run per sub-link, counted in valid samples, has the percentiles 43 min (p5), 108 min (p25), 203 min (p50), 380 min (p75), 704 min (p95) and 1069 min (p99), and a maximum of 1595 min (26.6 h, link 10116). It exceeds 1 h on 651 sub-links, 3 h on 390, 6 h on 197 and 12 h on 33. Because RSL moves in 1/3 dB steps, a quiet link can stay on one value for hours; the starting value of 6 h would mark 27 % of the sub-links, so it was raised to 12 h.

TSL changes of at least 1 dB occur on 221 sub-links, 3 693 618 in total; per sub-link the count has the percentiles 39 (p25), 70 (p50), 150 (p75) and 166 509 (p95) and a maximum of 313 953. TSL takes one value on 496 sub-links, two on 106, three on 81 and more on 35. The changes are mostly not isolated: the median interval between consecutive changes is 0.3 min. Two populations exist:

most sub-links with changes have tens to a few hundred (occasional 1 dB steps), and about 10 % alternate between two adjacent values almost every sample (Section 4.11).

The share of alive sub-links with missing RSL per 10 s step has the percentiles 0.14 % (p50), 0.7 % (p90), 1.8 % (p99) and 8.0 % (p99.9). Table 16 shows the periods found at four thresholds. Periods above 20 % lasting at least 1 min all reach 100 %, that is, the whole network was missing (13 June 2015 14:28 to 14:29, 22 August 2015 08:46:50 to 08:48:10 and five periods of 1 to 2 min on 29 August 2015 between 17:09 and 17:24). On 27 August 2015 5 to 10 % of the sub-links, up to 59 %, were missing together for about 4.3 h, which the starting threshold of 20 % reduces to 1.5 min. The high missing shares of 18 and 19 July 2015 (3.91 % and 3.88 % of all RSL) are not network gaps: among alive sub-links the share is ordinary (0.96 % on 19 July), and the totals come from sub-links that were mostly without data on those days (687 alive sub-links on 18 July against 714 usually). The number of sub-links without any data changes on 19 days, between 10 and 14. A threshold of 5 %, about three times the 99th percentile, was chosen; it gives the 93 periods of `network_gaps.csv`.

Table 16: Network-gap periods over JJA by threshold on the missing share of alive sub-links.

Threshold	Periods	Total time	Longest
> 5 %	93	318.8 min	197.2 min
> 10 %	57	19.5 min	2.7 min
> 20 %	32	13.3 min	2.0 min
> 50 %	21	10.8 min	2.0 min

The valid ranges of `out_of_range` lie outside the published extremes (Section 3.2) and are a safety net that flags no sample in OpenMRG.

4.3.5 Verification

Synthetic tests with injected events assert the exact positions of every flag: short spikes are flagged and nothing else is; a floor plateau is flagged, also across midnight; a flatline of 14 h is flagged and runs of 8 h and 2 h are not; a single TSL step is flagged; `network_gap` applies only to the shared gap; the status table, the summary and the gap tables are correct; L1 is unchanged and a rerun is identical; the L2 validator (`validate_flags_l2`) works; decoding and blocking masks round-trip; and the unit functions are correct.

A pilot ran the flags on five days (2 June, 18, 20 and 28 July, and 27 August 2015) and the examples of every flag were inspected visually in a gallery. Table 17 lists the flag counts. The pilot found 22 network-gap periods on these days, among them 197.17 min on 27 August 2015 from 11:56:20 (up to 58.7 % of the alive sub-links, a group of links numbered 102xx) and 61 min on the same day from 15:59. The highest blocking shares on the pilot days were 0.509 % (10046 `sublink_1`), 0.435 % (10163) and 0.414 % (10140), all from `rx_floor`. The inspection confirmed the following: the spike examples are single-sample dips of 5 to 20 dB on calm signals, mostly on short 37 to 38 GHz links (10120, 10058, 10113, 10253, 10095 at 0.17 km, 10252); `rx_floor` marks clipped outages on 10163, 10140 and 10046 on 28 July 2015 from 15:47 to 16:20 (flat at

−89 to −90 dBm for 15 to 18 min, TSL missing at the same time) and the half-dead sub-link 10074 at −93.7 dBm; `flatline` marks truly constant signals, for example 10071 at −43.2 dBm for 13 h 53 min.

Table 17: Flagged samples on the pilot days (728 sub-links × 8640 steps = 6 289 920 samples per day).

Day	miss. RSL	miss. TSL	floor	spike	TSL step	flatline	net. gap	blocking
2015-06-02	115 445	115 442	0	134	45 723	0	0	166 480
2015-07-18	245 933	237 293	0	38	44 343	2067	976	289 635
2015-07-20	135 149	126 525	5	4	40 209	9390	0	195 952
2015-07-28	155 208	141 129	586	22	34 015	1393	11 317	218 319
2015-08-27	230 216	229 816	0	53	39 881	0	91 877	293 369

4.3.6 Issues found and corrections

Four definitions were corrected during calibration and pilot: the floor definition, the flatline and network-gap thresholds (all described above), and the spike rule. In the first pilot the six strongest spikes were the sharp minima of rain fades (for example sub-links of links 10110 and 10009 at 16:02 on 28 July 2015, 20 to 25 dB below the rolling median for 1 to 3 samples): at a V-shaped minimum the samples on the two walls agree with each other, so the neighbour test passed. The added condition that both neighbours lie close to the rolling median (`neighbour_max_dev_db`) reduced the spike counts from 1107 to 53 samples on 27 August, from 92 to 22 on 28 July and from 283 to 134 on 2 June 2015, and a synthetic V-shaped fade now guards this case. The flatline duration was first measured in clock time including gaps; it marked a cluster of runs of about 23.7 h on links 10264 to 10278 that were in fact a few minutes of data on each side of a long outage with the same quantised value. It is now counted in valid samples.

Two further observations were recorded for later work and are not flags. On some links (for example 10047 and 10315 on 27 August 2015) TL rises by 10 to 15 dB during rain while RSL stays nearly constant: the transmitter raised its power, which resembles ATPC although it is documented as disabled (Section 3.2); attenuation must therefore be computed from TL and not from RSL alone on those links. A 32.17 min outage during the Torslanda storm (28 July 2015, from 15:46:20 UTC, up to 77.0 % of the alive sub-links) coincides with rain but was identified as a collector outage, since the affected sub-links had no fade 5 to 10 min before (median 0.1 to 0.2 dB below their daily median), were spread over the whole network, and 64 % of them lost TSL at the same instant.

4.4 Geometry

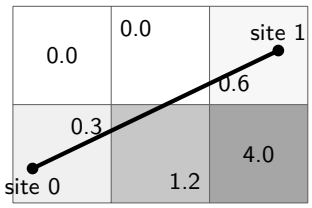
4.4.1 Purpose

Comparing a link with the rain requires the rain along its path: which radar cells the path crosses and which fraction of the path lies in each cell. It also requires the distance of every gauge from every path, the sites shared by several links, and the neighbouring links. All of this depends only

on positions and is computed once by the stage `geometry` (`src/cmlpipe/geometry/core.py` and `stage.py`).

4.4.2 Method

All link ends and gauges are projected to EPSG:3006 (`project_points`), so that lengths and distances are true metres. Link ends closer than a tolerance are merged into one site (`merge_sites`). The straight path of a link is intersected with the radar grid in the radar's own polar stereographic projection (`project_to_radar`, without datum shift), because only there are the cells exact 2 km squares; the grid is described by `RadarGrid` from the cell centres of the L1 radar files, which must all agree and must use the projection string of the configuration. The function `path_grid_weights` cuts the path at every crossing of a cell edge and assigns each piece to the cell that contains its midpoint, so that no piece is counted twice on an edge or a corner. The *path weight* w_i of cell i is the fraction of the path inside it (Figure 4); since fractions are independent of the map scale, the distortion of the projection does not affect them. The weights of a link sum to its *radar coverage*, the fraction of the path inside the grid.



Cells of 2 km; numbers are rain rates in mm/h.

The path is cut at every cell edge; each piece belongs to the cell of its midpoint. The weight w_i of a cell is the fraction of the path inside it.

R_{mean}: weighted mean over the cells with data (Eq. (4)); $R_{\max}$: largest crossed cell.

Figure 4: Path weights and radar rain along a link (schematic, values for illustration only).

The distance from a gauge to a link is measured to the path segment, not to its midpoint (`point_to_path`), together with the position of the closest point along the path (0 at `site_0`, 1 at `site_1`), the distance to the midpoint and to the nearest end. The azimuth of each path is computed in grid coordinates (`azimuth_deg`). The computed length is compared with the metadata length; a link is flagged as a length mismatch only if both tolerances are exceeded. Table 18 lists the parameters and Table 19 the outputs. The stage skips when the stage hash and its input signature (link ends, gauges, radar grid) are unchanged.

Table 18: Geometry parameters (keys under `geometry`).

Key	Value	Rationale
`crs_metric`	EPSG:3006	Metric projection for Sweden
`radar_proj_string`	polar stereographic	Projection of the radar grid
`site_merge_tol_m`	10 m	Ends of one mast
`gauge_match_max_m`	5000 m	Gauges kept per link
`gauge_flag_radii_m`	500, 1000, 2000 m	Distance classes in the match table
`neighbour_max_m`	5000 m	Neighbouring links
`length_mismatch_tol`	2 % and 30 m	Relative and absolute tolerance

Table 19: Outputs of the geometry stage.

File	Content
`sites.csv`	Sites: position, number of links and ends, largest offset of a merged end
`link_geometry.csv`	Per link: sites, computed length, length difference, azimuth, number of radar cells, coverage
`link_gauge_matches.csv`	852 link and gauge pairs up to 5 km: distance to path, position along path, distance to midpoint and nearest end, distance classes
`gauge_radar_pixel.csv`	Radar cell of every gauge and its distance from the cell centre
`link_neighbours.csv`	Pairs of neighbouring links, with shared sites
`path_grid_weights.nc`	Path weights (link, cell row, cell column, weight) with the grid definition

4.4.3 Verification

Synthetic tests check the distance of a point perpendicular to the midpoint, beyond either end and on the path; a horizontal path through three cells; a diagonal path through corners without double counting; a path along a cell edge; a path starting outside the grid; a path inside one cell; equality with the polygon-intersection lengths of `shapely` for generic paths; equality with the independent implementation of `poligrain` [5] on the same descending grid; the application of weights to a uniform field, to two cells with equal shares, to a field with one missing cell, to a single hot cell with two links and to a step without data; site merging at 5 m but not at 20 m; lengths in metres against the metadata in km; the tables; idempotence and skipping of the stage; and rejection of a different projection string.

Table 20 compares the outputs with independent values. Every link end coincided exactly with the other ends of its site, so the merging tolerance was not needed in practice: the sites are the distinct coordinate pairs of the metadata. Table 21 gives the number of links per site (up to 17, at site 25). Two links share both sites (10134 and 10135, 3555 m, 28.26 and 28.18 GHz). The sum of all lengths is 1044.6 km, against the sum over unique hops in Table 20. The computed lengths range from 142.2 to 15 234.5 m; the absolute difference from the metadata has a median of 0.46 m, a 95th percentile of 1.98 m and a maximum of 6.80 m (link 10201), with a small positive median bias of 0.39 m. The EPSG:3006 length differs from the WGS84 geodesic by a median relative $-6.2 \cdot 10^{-6}$, while the geodesic differs from the metadata by $2.1 \cdot 10^{-4}$, so the bias lies in the metadata lengths, whose computation is not documented.

The weights file has 1041 (link, cell) entries. A link crosses a median of 2 and at most 11 cells; the distribution is 62 links in 1 cell, 123 in 2, 82 in 3, 49 in 4, 28 in 5, 7 in 6, 7 in 7, 1 in 9, 4 in 10 and 1 in 11, and links longer than 2 km cross a median of 0.90 cells per km. The gauges Drakeg and GbgA lie in the same radar cell. Table 22 lists the number of links with a gauge within a given distance of the path; per gauge the counts within 1 km are Jarn 5, Torp 11, Bergsj 12, Torsl 9, Chalm 8, Tole 5, Barl 12, Drakeg 20, Lbom 12, Askim 3 and GbgA 12. An additional script recomputed the gauge matches and lengths from the raw metadata with a separate point-to-segment routine and a brute-force search over all link and gauge pairs and

Table 20: Acceptance of the geometry (reference values from [1] unless stated).

Quantity	**Pipeline**	**Reference**
Links	364	364
Sites (ends within 10 m merged)	418	418 nodes
Largest offset of a merged end	0.00 m	below 10 m
Unique hops	363	363
Length summed over unique hops	1041.0 km	1041 km
Convex hull of the sites	2855 km^2	about 2800 km^2
Links with a length mismatch	0	0
Largest relative difference from metadata length	$2.51 \cdot 10^{-3}$	below 0.02
Radar file coordinates against its grid	0.22 m	consistent
Links fully inside the radar grid (minimum coverage 1.000 000)	364	all
Weights against coverage, largest difference	$1.1 \cdot 10^{-16}$	0
Gauges inside the radar grid	11	11
Weights against `poligrain` 0.3.1, links 10227, 10115, 10045, 10283, 10201	$4.6 \cdot 10^{-14}$	0
Links with a City gauge within 1 km of the path	86	86 (independent implementation)

Table 21: Links per site (number of links: number of sites).

Links	1	2	3	4	5	6	7	8	9	10	11	17
Sites	296	58	23	18	7	3	3	4	1	1	3	1

confirmed that no row was missing; it was afterwards removed.

Table 22: Links with a gauge within a distance of the path.

Network	**$\leq$ 500 m**	**$\leq$ 1 km**	**$\leq$ 2 km**	**$\leq$ 5 km**
City	32	86	149	195
SMHI	5	12	35	108
Any	37	87	150	195

For the Torslanda link 10227 the computed length is 1791.8 m (metadata 1791.6 m) and the azimuth 66.0°; the gauge Torsl lies 203.4 m from the path at its end (position 1.000) and 782 m from the centre of its radar cell, and the path lies 95.08 % in that cell and 4.92 % in the neighbouring one. Pairs of neighbouring links number 1587 within 1 km, 3024 within 2 km and 9101 within 5 km; 906 pairs share a site and 998 pairs touch or cross.

A sensitivity test shifted all links relative to the radar grid by 200 m, about the offset caused by reading WGS84 coordinates as if they were on the Swedish RT90 datum (156 m at Gothenburg). Depending on the direction, the set of crossed cells changed for 72 to 103 of the 364 links (20 to 28 %), and on average 9.2 to 10.4 % of a path moved into another cell (medians 7.2 to 8.0 %, at most 100 % for a short link near a cell edge). A datum error of that size therefore matters for single short links, but changes the weights smoothly between neighbouring cells.

4.4.4 Issues found and corrections

No issue required a correction in this stage. The outputs were regenerated once with a clean code version so that their provenance refers to a committed state.

4.5 Aggregation of the link data

4.5.1 Purpose

The sources have different time steps (10 s, 1 min, 5 min, 15 min, 1 h). Aggregation places them on common bins of 1 min, 5 min, 15 min and 1 h so that they can be compared. Every bin length divides a day, so no bin crosses midnight and each day can be processed separately. This subsection covers the CML data; the gauges and the radar are covered in Section 4.6.

4.5.2 Method

The stage `aggregate_cml` (`src/cmlpipe/aggregate/core.py` and `stage.py`) reads one L1 day and its L2 flags and writes one file per resolution, `L3_products/cml/<res>/cml_<res>_<day>.nc`. A 10 s sample belongs to the bin that contains its label. The computation is vectorised: the data of a day are reshaped to (bins, samples per bin, ...) and reduced in one step (`bin_statistics`). For each variable only the samples without a blocking flag of its set (Table 11) are used (`blocking_mask`); TL therefore needs both RSL and TSL valid. Table 23 lists the statistics. A bin in which the number of valid samples is below $\lceil 0.8\,n \rceil$, with n the number of samples per bin (key `aggregate.min_valid_fraction`), is set to NaN for that variable (`min_count`); the count `n_valid_<var>` is always stored, so that a different rule can be applied later. The bitwise OR of all flags in the bin (`flags_or`) and the number of TSL-step samples (`n_tsl_step`) are stored as well. The standard deviation uses the population form. Every product is checked by `validate_l3` (dimensions, a regular interval-start time axis, time attributes, units, counts).

Table 23: CML statistics per bin (key `aggregate.stats`).

Variable	5 min, 15 min, 1 h	1 min
`tl`	mean, median, min, max, std	mean, median, min, max, std
`rsl`	mean, median, min, max, std	mean, median, min, max
`tsl`	mean, min, max	mean, min, max

The storage type was chosen by projection from the pilot days. Float32 needs 1.69 GB for the summer (1.14 GB at 1 min, 0.35 GB at 5 min, 0.14 GB at 15 min, 0.06 GB at 1 h); 16-bit integers with a scale of 0.01 dB would need 0.88 GB with a maximum error of 0.0050 dB. Since 1.69 GB is below the limit of 6 GB set in advance (396 GB were free), float32 was kept (key `aggregate.encoding.db_dtype`). The stage skips a day only when every file carries the current stage hash and was made from L1 and L2 files with the current stage hashes.

4.5.3 Verification

The aggregation tests use a synthetic world of two days that is processed end to end. They check exact bin statistics; that blocking flags are excluded and informative flags are not; the 80 % rule at its boundary (in a 15 min bin of 90 samples, 72 valid samples, 80.0 %, keep the value and 71, 78.9 %, do not, while the valid count is stored in both cases); that an unflagged spike moves the mean but not the median; that TL needs RSL and TSL; that `flags_or` is a bitwise OR; that aggregating days separately equals aggregating the two-day series; that `min_count` is exact; the product axes and that the inputs are unchanged; skipping and `--force`; equality of one and four workers; and the validator. One test recomputes a real pilot day.

The acceptance report recomputed the statistics of three sub-links on 28 July 2015 with plain `numpy`: the maximum differences of the mean, median, minimum and maximum of TL were $3.6 \cdot 10^{-6}$, $3.1 \cdot 10^{-6}$, $3.1 \cdot 10^{-6}$ and $3.1 \cdot 10^{-6}$ dB at 1 min (limit 10^{-4} dB, float32), and the valid counts were equal. Table 24 shows the share of NaN bins on the pilot days, and Table 25 the coverage by all three sources. With four workers one day took 11.5 s on average (8.8 to 12.4 s), which projected about 4 min for all days.

Table 24: NaN shares of the CML statistics on the five pilot days.

Resolution	Bins	NaN, 80 % rule	NaN, no sample	Days all NaN
1 min	5 241 600	0.53 %	3.34 %	2.66 %
5 min	1 048 320	0.62 %	3.31 %	2.66 %
15 min	349 440	0.74 %	3.29 %	2.66 %
1 h	87 360	0.83 %	3.27 %	2.88 %

Table 25: Coverage of link bins on the pilot days.

Resolution	Link bins	Valid TL	Valid radar	Gauge $\leq$ 1 km	All three
1 min	2 620 800	96.2 %	0.0 %	23.0 %	0.0 %
5 min	524 160	96.1 %	99.7 %	23.0 %	22.2 %
15 min	174 720	96.0 %	100.0 %	23.2 %	22.5 %
1 h	43 680	95.9 %	100.0 %	23.2 %	22.5 %

4.5.4 Issues found and corrections

No issue required a correction in this stage.

4.6 Rain from gauges and radar

4.6.1 Purpose

The rain references serve three purposes: to define when it rained (Section 4.7), to label each link wet or dry (Section 4.12), and to provide the rain along the path, $\bar{R}$ in Eq. (2), against which the attenuation laws will be learned. Gauges measure the rain that falls into one funnel, accurately but at a single point. The radar observes the whole network from above but measures

reflectivity, from which rain must be estimated. The processing spans the stages `ingest_gauges`, `ingest_radar`, `geometry`, `aggregate_gauges` and `aggregate_radar`.

4.6.2 Gauges

After ingest the gauges hold amounts in mm per native step, labelled by the start of the step. The stage `aggregate_gauges` (function `aggregate_gauges`) adds the amounts of all native steps inside a bin; amounts are always summed and never averaged, and the sums are computed in float64 because a float32 sum over the season has a rounding error of about $5 \cdot 10^{-3}$ mm. A bin is NaN unless every native step in it is valid, because a sum with a missing step would be too low. The rain rate in mm/h is the amount divided by the bin length. For the City gauges the highest 1 min rate inside each bin is kept (`peak_1min_rate_mm_h`); for the SMHI gauge the worst quality code in the bin (`qc_worst`). The SMHI gauge is never divided into intervals shorter than its native 15 min, which would invent information, and it is never used to define events or wet periods.

4.6.3 Radar

Each cell value v of the radar file is decoded to reflectivity as $\mathrm{dBZ} = 0.4\,v - 30$, with 255 marking a missing cell. Reflectivity is converted to rain rate with the Z-R relation stored in the file,

$$Z = 10^{\mathrm{dBZ}/10}, \qquad Z = 200\,R^{1.5} \;\Rightarrow\; R = (Z/200)^{1/1.5}, \tag{3}$$

with Z in $\mathrm{mm}^6\,\mathrm{m}^{-3}$ and R in mm/h (`zr_rain_rate`). The values $a = 200$ and $b = 1.5$ are the SMHI summer values; the Marshall-Palmer relation would use $b = 1.6$ [1]. When a file does not provide the relation, these values serve as fallback. The stage `ingest_radar` stores dBZ and rain rate in monthly files.

Before rain is accumulated, every rate below 0 dBZ is set to zero (`floor_rain`). The radar never reports exactly zero: cells without echo hold the lowest level, and 0 dBZ corresponds to about 0.03 mm/h. Summed over the summer this background would add rain that did not fall: the domain-mean accumulation is 237.6 mm without and 236.7 mm with the threshold, a difference of 0.9 mm. The threshold sets 90.8 % of the valid cell values to zero.

Instantaneous radar stamps are placed on the native grid of the period (`stamps_to_grid`, Section 3.6). The rain along a link is computed with the path weights of Section 4.4 (`apply_weights`). With w_i the weight of cell i and $R_i(t)$ its rain rate, the path-averaged rain rate is

$$R_{\mathrm{mean}}(t) = \frac{\sum_{i \in \mathcal{V}(t)} w_i\, R_i(t)}{\sum_{i \in \mathcal{V}(t)} w_i}, \tag{4}$$

where $\mathcal{V}(t)$ is the set of crossed cells with data at time t. R_{mean} is the radar estimate of $\bar{R}$ in Eq. (2). The stage also stores, per link and stamp, the weighted standard deviation `R_std` (population form), the rate of the wettest crossed cell `R_max`, the `wet_fraction` (the share of the

valid path weight in wet cells) and the `valid_weight_fraction` (the share of the path weight with data). A stamp is used for a link only when a minimum share of its path weight has data. Only the bounding box of the crossed cells is read, one day at a time, so a monthly file is never loaded whole.

The 5 min values are then aggregated to 15 min and 1 h (`aggregate_stamps`): the mean of the valid stamps, or their maximum for `R_max`, with `n_valid_stamps` stored. A bin is NaN when too few of its stamps are valid; with the threshold of Table 26 that means fewer than 2 of 3 stamps at 15 min or 8 of 12 at 1 h. The radar is never interpolated to 1 min. For each gauge the stage also stores the rate of the cell containing it (`R_pixel`) and the mean of the surrounding 3×3 cells (`R_3x3`), so that radar and gauges can be compared. Table 26 lists the parameters.

Table 26: Parameters of the rain references.

Key	Value	Rationale
`aggregate.gauges.min_valid_fraction`	1.0	A sum with a missing step is too low
`aggregate.gauges.city_resolutions`	1 min to 1 h	Native 1 min
`aggregate.gauges.smhi_resolutions`	15 min, 1 h	Native 15 min
`radar.zr_fallback`	$a = 200$, $b = 1.5$	Z-R relation of the file
`aggregate.radar.min_dbz`	0 dBZ	Removes the no-echo background
`aggregate.radar.wet_threshold_mmh`	0.1 mm/h	A cell above this rate is wet
`aggregate.radar.min_valid_weight_fraction`	0.5	Half of the path must have data
`aggregate.radar.min_valid_fraction`	0.6	Valid stamps required per bin
`aggregate.radar.resolutions`	5 min to 1 h	No radar data below 5 min
`aggregate.radar.time_chunk`	288	Stamps read at once (one day)

4.6.4 Verification

Tests check gauge sums and the strict rule, that the SMHI gauge is not disaggregated, that the radar threshold sets values to zero instead of producing background rain, the alignment of stamps and the 15 min mean, and the radar and gauge products of the synthetic world. The acceptance report found that the City totals of the 9 gauges without gaps equal the L1 totals at every resolution to within $1.6 \cdot 10^{-6}$ mm (limit 10^{-3} mm); that the SMHI total remains 255.90 mm at 15 min and 1 h; that the missing block of Barl lies inside one gap from 1 June 2015 21:59 to 3 June 2015 21:58, so that only the partial bins at its edges became NaN and, since they were dry, no rain was lost; that no SMHI product exists below 15 min and no radar product at 1 min; and that `R_mean` of link 10227 at 5 min equals a manual weighted mean over all 26 495 stamps to within $1.3 \cdot 10^{-6}$ mm/h. Table 27 compares the radar cell above each gauge with the gauge; since the radar is adjusted to gauges (Section 3.4), the comparison is information only.

The radar total is below the gauge total at 10 of the 11 gauges, by 3 to 24 %; the exception,

Table 27: Radar cell against gauge over JJA (radar total from the 5 min cell means of the valid bins).

Gauge	Network	Gauge total	Radar total	Radar NaN	r 15 min	r 1 h
Jarn	City	236.6 mm	223.0 mm	1.43 %	0.73	0.80
Torp	City	271.2 mm	233.9 mm	0.56 %	0.69	0.76
Bergsj	City	301.1 mm	228.3 mm	0.56 %	0.72	0.81
Torsl	City	272.8 mm	224.3 mm	1.43 %	0.61	0.75
Chalm	City	277.2 mm	212.7 mm	0.56 %	0.53	0.61
Tole	City	250.8 mm	206.5 mm	1.43 %	0.60	0.72
Barl	City	231.5 mm	218.4 mm	1.43 %	0.57	0.74
Drakeg	City	185.4 mm	238.2 mm	0.56 %	0.71	0.72
Lbom	City	227.6 mm	220.6 mm	0.56 %	0.66	0.78
Askim	City	244.6 mm	224.3 mm	1.43 %	0.61	0.74
GbgA	SMHI	255.9 mm	238.2 mm	0.56 %	0.56	0.68

Drakeg, is a tipping-bucket gauge that shares its radar cell with GbgA. In the Torslanda event the radar cell above the gauge did not record the first peak [1].

4.6.5 Issues found and corrections

No issue required a correction in the aggregation of the references; the correction of the radar ingest is described in Section 4.2.

4.7 Rain events and dry periods

4.7.1 Purpose

The research requires the periods in which rain fell, to study fades and drying, and the periods that were certainly dry, to estimate baselines. These periods are defined from the radar and the City gauges only and never from the CML signals, because selecting training data with the model's own input would be circular. The SMHI gauge is excluded from the definition and serves as an independent check.

4.7.2 Method

An *event* is a period in which it rained somewhere over the link network. Figure 5 shows how the stage `events` (`src/cmlpipe/events/core.py` and `stage.py`) finds events and dry periods, and Table 28 lists the parameters. The *domain* is the set of radar cells whose centre lies within a buffer distance of any link path (`domain_mask`); it contains 720 of the 1776 grid cells, 2880 km^2. `build_reference` computes, for every 5 min bin of the period, the *network reference series*: the share of domain cells with radar data, the *wet-area fraction* (the share of valid domain cells with a rain rate at or above the wet threshold), the number of wet cells and the domain-mean rate (`radar_domain_series`), and for the 10 City gauges the number of valid and wet gauges and the largest rate (`gauge_series`). `classify` then labels each bin:

- A bin is *wet* if the radar is valid and the wet-area fraction reaches its threshold, or if at least one City gauge reaches the gauge threshold.
- A bin is *dry* if it is not wet, the radar is valid and no domain cell is wet.
- A bin is *unknown* otherwise, that is, when the radar is missing or only a few cells (1 to 7) are wet. Unknown never counts as dry, and a gauge with a missing bin counts as unknown for that gauge.

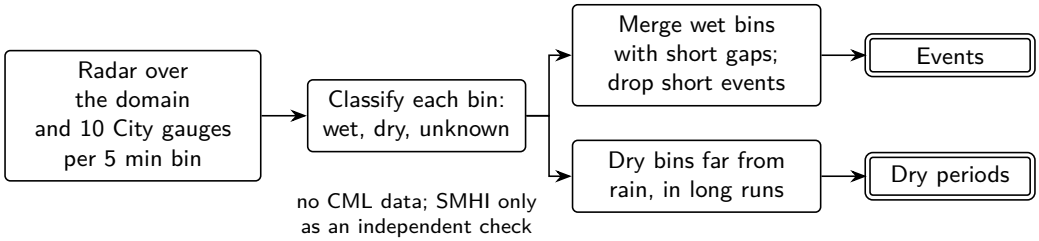


Figure 5: Definition of events and dry periods. Double frames mark frozen tables.

Wet bins separated by less than the minimum dry gap are merged into one event, so that one storm with short pauses forms one event, and events shorter than the minimum duration are dropped (`find_events`). An event closer than one minimum gap to the start or end of the period is kept and marked (`touches_edge`). The identifier is "E" followed by the start time (`event_id`). *Dry periods* are runs of dry bins of a minimum length that lie more than a margin from any wet bin (`certified_dry`, using `dilate`). For every event `context` computes the gap to the neighbouring events and the dry time before and after it, up to a cap.

Table 28: Event and dry-period parameters (keys under `events`).

Key	Value	Rationale
`resolution`	5 min	Native radar step
`domain_buffer_m`	3000 m	Cells that can influence a link
`wet_threshold_mmh`	0.1 mm/h	Wet radar cell
`radar_min_valid_cell_fraction`	0.5	Radar valid in a bin
`min_wet_area_fraction`	0.01	At least 8 wet cells (calibration)
`city_gauge_wet_amount_mm`	0.1 mm	Wet City gauge
`min_gauges_wet`	1	One wet gauge suffices
`min_dry_gap_h`	2 h	Calibrated (Table 30)
`min_event_duration_min`	15 min	Drops short isolated wet runs
`dry_max_wet_area_fraction`	0.0	Dry requires no wet cell
`dry_margin_min`	30 min	Distance of a dry period from rain
`dry_min_duration_h`	6 h	Minimum dry period
`dry_context_cap_h`	48 h	Longest dry context counted
`dry_stats_min_days`	5	Days required for dry statistics

The table `events.csv` has one row per event with its timing and context (start, end, duration, gaps, dry time before and after, number of merged wet runs, edge mark), radar summaries (peak and mean domain rate, largest wet-area fraction, domain total), the total of every City gauge with their mean and maximum, the largest 5 min and 1 min City rates and their gauges, the SMHI total for information, the number of unknown bins, the radar missing share, the minutes of

network gaps, minimum, maximum and mean of the weather variables, the month, the *intensity class* (terciles of the largest 5 min City rate: light, moderate, heavy) and the split. The table `dry_periods.csv` lists the start, end, duration and radar valid share of each dry period.

The stage also computes dry-weather statistics of every sub-link (`dry_stats`, `sublink_dry_stats.csv`) over the bins of the dry periods only: the median, mean, standard deviation, scaled MAD, interquartile range and 1st and 99th percentiles of TL, the median within-bin standard deviation (the 10 s noise), the corresponding RSL values, the diurnal range and the drift of the daily medians. These statistics read CML data, but only after the periods have been defined. To read the CML data of all 92 days quickly, `build_core` writes a *consolidated core file* per resolution (`cml_5min_core.nc`, 174 MB, with `tl_mean`, `tl_std`, `n_valid_tl`, `rsl_mean`, `rsl_std` and `flags_or`; later a 1 h core file for the link cards), read with `netCDF4`. The reference series is written to `network_reference_5min.nc` with the event identifier of every bin, and a sensitivity grid to `event_calibration.csv`.

4.7.3 Calibration

The definition was calibrated with the script `events_report.py` (archived). Over the bins with valid radar the wet-area fraction has the percentiles 0.0, 1.9, 24.0, 51.6 and 98.6 % (p50, p75, p90, p95, p99) and is above zero in 34.1 % of the bins; in bins without a wet City gauge the values are 0.0, 0.4, 9.2, 20.4 and 52.1 % (above zero in 28.3 %). The radar was missing for all domain cells in 0.44 % of the bins and for some cells in 1.00 %, and valid in 99.44 %. Table 29 gives the number of wet domain cells in bins without a wet City gauge.

Table 29: Bins with valid radar and no wet City gauge, by number of wet domain cells.

Wet cells	0	1	2	3	4 to 7	8 to 20	21 to 50	> 50
Bins	17 221	420	340	246	706	1067	1250	2768

A radar noise floor (clutter or residual noise) would appear as cells that are wet repeatedly in dry weather or as an excess of isolated wet cells. No such floor was found: the wet frequency per domain cell ranges from 6.25 to 7.85 % (median 7.12 %, maximum over median 1.10); stamps with only 1 to 3 wet cells are 1042 of 26 496 (3.9 %), and the cell most often wet in them is wet in 34 (median cell 2); isolated wet cells (no wet neighbour among the 8 around) are 0.82 % of wet cell values; and the counts fall smoothly from 1 cell upward. The threshold of 1 % of the domain lies above the 1 to 3 isolated cells that form the only plausible floor.

Of the wet bins, 5092 (68.1 %) are wet by the radar only, 255 (3.4 %) by the City gauges only and 2126 (28.4 %) by both. Radar-only bins are mostly rain elsewhere in the domain, away from the gauges; gauge-only bins are rain the radar misses or places below the wet threshold on less than 1 % of the domain. Tables 30 and 31 give the sensitivity of the definition. At a fixed gap the number of events hardly depends on the area threshold (82 to 95 events at 2 h over a 25-fold range), while the gap matters. The share of wet bins is 33.1, 30.8, 28.2, 25.8 and 21.4 % for the five area thresholds. The initial values were kept, since the data gave no reason to change them;

the dry context of every event is stored so that a stricter selection can be made per analysis.

Table 30: Number of events by area threshold and minimum dry gap.

Area threshold	1 h	2 h	3 h	6 h
0.002	124	95	83	53
0.005	128	93	80	55
0.01	119	89	78	58
0.02	116	89	75	58
0.05	107	82	67	54

Table 31: Total event hours by area threshold and minimum dry gap.

Area threshold	1 h	2 h	3 h	6 h
0.002	818	885	932	1125
0.005	758	837	892	1072
0.01	690	763	817	978
0.02	632	699	767	913
0.05	536	601	670	811

4.7.4 Results

The definition gives 89 events (27 in June, 27 in July, 35 in August) after 56 wet runs shorter than the minimum duration were dropped. Event durations have the percentiles 0.7 h (p10), 4.1 h (p50) and 19.5 h (p90) and a maximum of 49.2 h; the mean City total per event 0.0, 0.0 and 9.9 mm (p10, p50, p90) with a maximum of 26.5 mm; the largest 5 min City rate 0.0, 1.2 and 24.0 mm/h with a maximum of 82.8 mm/h. The intensity classes contain 30 light, 30 moderate and 29 heavy events. Two events touch the period edge. 72 events have at least 3 h and 53 at least 6 h of dry time before them, and the same numbers after them. 79 events are merged from more than one wet run, at most 14. There are 62 dry periods with 921 h in total, the longest 90.6 h. Of the summer, 41.7 % lies in dry periods, 65.0 % is in the dry state and 6.8 % in the unknown state, and 34.5 % of the time lies inside events.

4.7.5 Verification

Synthetic tests check that wet runs 90 min apart merge with a 2 h gap and 3 h apart do not, that unknown counts as not wet for merging, that short runs are dropped and counted, that an event at the period start is kept and marked, the identifier format, that unknown is never dry, that the dry margin removes 30 min, that events from gauges only and from radar only are found, that missing radar is unknown and a small wet area is not dry, that the context counts dry and not unknown time, that the dry statistics use only dry bins, and that the definition reads neither SMHI nor CML data; the stages are also run end to end.

The acceptance report checked the definition against independent references. The Torslanda rain of 28 July 2015 (Section 3.5) lies inside one event, `E20150727_2140`, from 27 July 21:40 to 29 July 12:50 (39.2 h), with no Torsl rain outside events. The wettest SMHI day, 17 June

2015 (Table 39), lies in the event `E20150617_0705`, which contains all its rain. Of the SMHI JJA total, 100.00 %, and of its 488 wet 15 min steps, 100.00 %, fall inside events, although the SMHI gauge was not used in the definition. Of the City total, 99.74 % falls inside events; the remaining 6.6 mm (sum of the 10 gauges) belongs to dropped short runs. The wet share of the domain indicator (28.2 % of 5 min bins, or 31.1 % of 15 min steps containing a wet bin) is much higher than the SMHI point value (Table 39), because the indicator means "rain somewhere in 2880 km^2"; the comparable point value, the median wet frequency of a single radar cell, is close to it.

As information only, the CML response was examined after the definition: the share of sub-links whose largest 5 min `tl_mean` in an event exceeds the median of the 3 h before by at least 3 dB has the percentiles 0.1, 1.2 and 94.5 % over events (p10, p50, p90), against 0.4, 0.9, 3.4 and 5.1 % (p50, p90, p99, maximum) in 300 random 3 h windows inside dry periods. In 52 of the 89 events fewer than 5 % of the sub-links respond; 49 of them have a mean City total below 0.1 mm and a radar domain total below 1 mm. They are drizzle or small echoes covering at least 1 % of the domain, valid by the definition; at 0.1 to 1 mm/h the specific attenuation at 20 to 40 GHz is well below 1 dB/km, so no 3 dB rise is expected. They were not removed, because thresholds are not tuned on the CML data; they are labelled by the significance stage (Section 4.10). The stage `events` ran in 14.7 s; reading one variable of all sub-links and days from the core file took 0.5 s.

4.7.6 Issues found and corrections

No issue required a correction in this stage.

4.8 Train, validation and test split

4.8.1 Purpose

Models must be evaluated on data they were not trained on: on events not seen in training, on links not seen in training, or both. The split is drawn once and frozen, so that every analysis uses the same sets.

4.8.2 Method

The stage `split` assigns events to train, validation and test, stratified by intensity class (`split_events`), and links to train and test, stratified by length class and frequency band of sub-link 1 (`split_links`); Table 32 lists the proportions and classes. The random generator permutes the sorted identifiers within each stratum; the counts per stratum are allocated by the largest remainder (`_allocate`). Links that share both sites (a *hop*) are kept in the same set. Links whose two sub-links were both unusable in the first status (Section 4.3) are excluded. The stage writes `split_events.csv` and `split_links.csv` and adds the event split to `events.csv` (`annotate_events`). The five tables `events.csv`, `dry_periods.csv`, `split_events.csv`,

`split_links.csv` and `event_links.csv` (Section 4.9) are *frozen*: the stages refuse to overwrite them unless `--force-split` or `--force` is given, and `--force` does not imply `--force-split`.

Table 32: Split parameters (keys under `split`).

Key	Value	Rationale
`seed`	42	Reproducible draw
`event_fractions`	0.6, 0.2, 0.2	Train, validation, test events
`link_fractions`	0.8, 0.2	Train, test links
`length_classes_km`	1, 3 km	Classes below 1 km, 1 to 3 km, above 3 km
`frequency_split_ghz`	25 GHz	Bands below 25 GHz and from 25 GHz

The intended evaluation uses train events on train links for fitting (tuning on validation events), test events on train links for unseen events, train events on test links for unseen links, and test events on test links for both.

4.8.3 Verification and results

Tests check the sizes, the stratification and the determinism of the event split, that parallel links stay together and excluded links are excluded, that the split is frozen, and that an event without a split row is an error. The split gives 53 train, 18 validation and 18 test events (heavy 17, 6, 6; light 18, 6, 6; moderate 18, 6, 6) and 285 train, 71 test and 8 excluded links. No hop is split between train and test, and rerunning the stage left the files unchanged.

4.8.4 Issues found and corrections

No issue required a correction. After the quality status was revised (Section 4.11) the split was not redrawn.

4.9 Event pairs

4.9.1 Purpose

An *event pair* is one event combined with one link: the record of how one link behaved during one rain event. Event pairs are the basic samples for learning the attenuation laws.

4.9.2 Method

The stage `event_links` (`event_links_table`) writes all pairs, one row per event and link (89 events × 364 links = 32 396 rows), to `event_links.csv`. Figure 6 shows the three windows of a pair, and Table 33 lists the parameters. Each row holds the link properties (length, frequencies, polarisation, number of radar cells); the radar along the path during the event (total, peak rate, wet minutes, mean wet fraction, largest `R_max`, mean valid weight fraction); the *matched gauge*, the nearest gauge within a maximum distance of the path, with its network, distance, position along the path, total, largest rate and its interval (City at 5 min, SMHI at 15 min);

the number of gauges within 1 km; for each sub-link the valid share of the 5 min `tl_mean` bins in each window, with the minimum over both sub-links; the shares of `rx_floor`, `spike` and `network_gap` in the event; the status of both sub-links; the indicators `cml_ok_event`, `cml_ok_before` and `cml_ok_after` (valid share at least the minimum); `has_reference_gauge` (a gauge within 1 km); and the event and link splits. The stage `event_links_extra` adds, in `event_links_flags_extra.csv`, the shares of `flatline` and `tsl_step` and the number of TSL steps per pair.

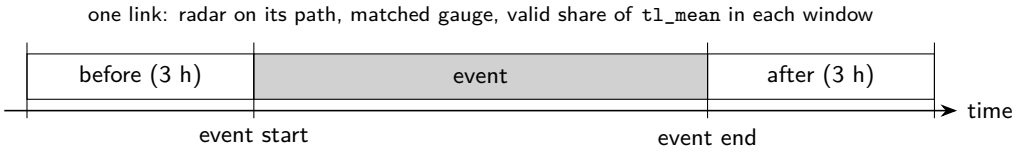


Figure 6: Windows of an event pair.

Table 33: Event-pair parameters (keys under `event_links`).

Key	Value	Rationale
`pad_before_h`	3 h	Window before the event
`pad_after_h`	3 h	Window after the event
`gauge_match_max_m`	2000 m	Matched gauge
`reference_gauge_max_m`	1000 m	Reference gauge near the path
`min_valid_fraction`	0.8	Valid share for `cml_ok_*`

For modelling, the method `CMLData.event_pairs` (Section 7.2) selects pairs from these tables. By default it keeps significant events only (Section 4.10) and drops a pair when no sub-link of the link has the status good or degraded (reason `unusable`), when RSL was frozen for more than half of the event (`flatline`), or when the better usable sub-link has less than 80 % valid 5 min bins in the event (`low_valid`). The selection by split accepts an event split or a pair of event split and link split. With `return_dropped=True` the dropped pairs are returned with all reasons that applied.

4.9.3 Verification and results

A synthetic release of two events and four links tests the filters and the reasons, and a real-data test checks the result on `openmrg_v1`. With the defaults, 12 289 of the 12 740 significant pairs are kept and 451 dropped: 280 for `unusable` and `low_valid`, 163 for `low_valid` and 8 for `flatline` (six links in `E20150828_2145`, link 10329 in `E20150603_1500` and link 10076 in `E20150829_1445`). The stage `event_links` ran in 28.5 s; the table is 8.30 MB.

4.9.4 Issues found and corrections

No issue required a correction. The table was written as CSV during development because `pyarrow` was not yet installed; the release provides it also as Parquet.

4.10 Significance and wet runs

4.10.1 Purpose

Many events cause no measurable attenuation (Section 4.7), so events are labelled by their rain amount with a physical rule that is not tuned on the CML data. The rule is kept in its own configuration section, so that the stage hash of `events` does not change.

4.10.2 Method

The stage `significance` (`src/cmlpipe/inventory/core.py`, `significance`) labels an event *significant* if the mean City total, the domain-mean radar accumulation or the largest 5 min City rate reaches its threshold, and *heavy* if, in addition, a total or the rate reaches the heavy threshold; all other events are labelled drizzle only (`none_drizzle`). The reason that applied is stored. `event_runs` lists the *wet runs* inside each event, the uninterrupted wet stretches before merging, with their totals and gaps; a run is significant by the same rule with both totals lowered to the run threshold. `wet_core` gives the *wet core* of each event, from the first to the last significant run. Table 34 lists the parameters; the outputs are `event_significance.csv` and `event_runs.csv`.

Table 34: Significance parameters (keys under `event_significance`).

Key	Value	Rationale
`city_total_mean_mm`	1.0 mm	Mean City total
`radar_domain_total_mm`	1.0 mm	Domain-mean radar accumulation
`city_max_5min_rate_mmh`	5.0 mm/h	Largest 5 min City rate
`heavy_total_mm`	10.0 mm	Heavy event total
`heavy_rate_mmh`	30.0 mm/h	Heavy event rate
`run_total_mm`	0.5 mm	Significant wet run
`min_significant_per_split`	6	Warning threshold for validation and test

4.10.3 Verification and results

Tests check the rule and the classes, the extraction of runs and the wet core, and that the stage keeps the frozen tables unchanged. The result is 9 heavy, 26 significant and 54 drizzle-only events, with 368 wet runs; the number of runs of every event equals its merged count in `events.csv`. Significant events number 11 of 27 in June, 12 of 27 in July and 12 of 35 in August; all 29 heavy-intensity events are significant, as are 5 of 30 moderate and 1 of 30 light events. Per split, 18 train, 7 validation and 10 test events are significant, and 8, 0 and 1 are heavy; the validation set has no heavy event because the split is stratified by intensity class and not by this label. The Torslanda event `E20150727_2140` consists of six runs (Table 35); the afternoon rain at Torsl lies inside run R02, which also contains the morning shower, because the network was wet without a break in between. Its wet core is 34.4 h.

Table 35: Wet runs of the Torslanda event E20150727_2140.

Run	Start	End	Minutes	City mean	Max 5 min	Radar	Significant
R01	07-27 21:40	07-28 01:30	230	0.04 mm	2.4 mm/h	0.16 mm	no
R02	07-28 01:35	07-28 19:05	1050	11.61 mm	82.8 mm/h	7.27 mm	yes
R03	07-28 19:30	07-28 20:05	35	0.01 mm	1.2 mm/h	0.00 mm	no
R04	07-28 20:10	07-29 12:00	950	11.47 mm	56.4 mm/h	11.21 mm	yes
R05	07-29 12:05	07-29 12:30	25	0.00 mm	0.0 mm/h	0.00 mm	no
R06	07-29 12:45	07-29 12:50	5	0.00 mm	0.0 mm/h	0.00 mm	no

4.10.4 Issues found and corrections

No issue required a correction.

4.11 Inventory and quality status

4.11.1 Purpose

Before links and events are selected for research, every sub-link needs a documented quality assessment. The stage `inventory` (`src/cmlpipe/inventory/stage.py`) collects it and assigns a status with a reason. It only adds information: the frozen tables and L1, L2 and L3 are read and never written.

4.11.2 Method

The stage reads the daily QC summary, the period statistics and histograms, the dry statistics, the split, the geometry tables, the network reference series, the 5 min core file and the per-day 5 min files (only `n_tsl_step`). It computes per sub-link (`tables/inventory/sublinks.csv`): the completeness of RSL, TSL and TL over JJA and per month (`completeness`); the share of samples not blocked; the share of every flag, and in particular `flag_share_blocking_present`, the share of samples with valid RSL and TSL that a blocking flag still removes; first and last valid day; the smallest RSL step and the number of distinct RSL and TSL values from the histograms (`quantization`); the TSL behaviour (`tsl_behaviour`); TSL steps inside events and dry periods (`tsl_steps_wet_dry`); the dry-weather noise (standard deviation of the 5 min TL means in dry periods, and the median 5 min standard deviation as 10 s noise); and the *dry baseline range*, the difference between the 90th and the 10th percentile of the medians of TL over the dry periods with at least 50 % valid bins (NaN with fewer than 5 periods; `dry_period_medians`, `dry_baseline_range`). Per link (`links.csv`) it summarises both sub-links and adds the best status, the gauges within 250, 500 and 1000 m, the nearest gauge at any distance (overall, City and SMHI), the neighbours within 1 and 5 km, and whether the link lies in a single radar cell. Per gauge (`gauges.csv`) it gives completeness per month, totals, wet fraction and the comparison with its radar cell. Table 36 defines the TSL classes, applied in this order.

The 1 % limit for `toggling` is placed in a gap of the distribution: the share of samples with a TSL change, on a logarithmic scale, is bimodal (Table 37, the 496 sub-links without changes

Table 36: TSL behaviour classes (keys under `inventory.tsl`).

Class	Rule	Sub-links
`no_tsl`	No valid TSL sample	11
`toggling`	TSL changes in more than 1 % of the valid samples	24
`constant`	At least 99 % of samples at one value and at most 0.1 changes per day	523
`rare_steps`	At most 2 changes per day	147
`frequent_steps`	More than 2 changes per day, in at most 1 % of the samples	23

excluded). Most sub-links change in less than 10^{-3} of their samples (occasional 1 dB steps); a separate group changes in 3 to 40 %. The ranges of $\log_{10}$ of the share are -5.71 to -3.65 for `rare_steps`, -3.62 to -2.43 for `frequent_steps` and -1.73 to -0.40 for `toggling`. The descriptive indicator `tsl_follows_fades_descriptive` (at least 5 TSL steps in events and a step rate in events at least 3 times the rate in dry periods) is true for 156 sub-links (136 `rare_steps`, 15 `frequent_steps`, 5 `constant`); since ATPC is documented as disabled, a likely cause is the temperature-driven 1 dB steps occurring more often in rain. It is never used for quality control or splitting.

Table 37: Sub-links by $\log_{10}$ of the share of samples with a TSL change (bin start).

Bin	−6.0	−5.5	−5.0	−4.5	−4.0	−3.5	−3.0	−2.5	−2.0	−1.5	−1.0	−0.5
Count	13	15	14	75	61	14	4	1	4	1	16	3

The *quality status* (`quality_status`, `quality_status.csv`) is assigned by the first rule that applies (Table 38); the reason string lists every rule that fired. Two rules compare a sub-link with the 95th percentile of the network (0.81 dB for the dry noise and 1.71 dB for the baseline range), so they mark about 5 % of the sub-links by construction; they identify the noisiest baselines relative to this network, not an absolute physical limit.

Table 38: Quality-status rules (keys under `inventory.status`) and the number of sub-links where each fires.

Status	Rule	Fires
unusable	TL completeness over JJA below 50 %	20
unusable	More than 20 % of present samples blocking-flagged	0
unusable	`rx_floor` above 20 %	0
degraded	A month with TL completeness below 80 %	7
degraded	`rx_floor` above 5 %	0
degraded	`spike` above 0.1 %	0
degraded	`flatline` above 10 %	0
degraded	`tsl_step` above 1 %	24
degraded	Dry-weather TL noise above the network 95th percentile	36
degraded	Dry baseline range above the network 95th percentile	36
good	None of the above	641

4.11.3 Verification and results

Tests check the completeness from the daily summary, the TSL classes, the status classification, the quantisation from the histograms and the link filters. An independent recomputation of the completeness of RSL, TSL and TL and of the share not blocked for five random sub-links from L1 and L2 (10211/1, 10227/2, 10249/1, 10327/1, 10343/1) agreed to within $1.1 \cdot 10^{-16}$. The result is 641 good, 67 degraded and 20 unusable sub-links. The caps for `spike` and `rx_floor` do not fire: the largest shares on a usable sub-link are 0.067 % and 0.055 %. The `flatline` cap does not fire either, but only narrowly: the largest share is 9.6 % and 6 sub-links exceed 5 %. The `tsl_step` cap fires on exactly the toggling sub-links. Links 10074 and 10034 stop reporting (last valid days 17 July and 10 June 2015, TL completeness 49.1 % and 10.1 %) and are unusable. During significant events, 8 of 12 390 pairs of usable links had RSL frozen for more than half of the event (8 links in 3 events). Length and frequency are correlated: the Pearson correlation of $\log_{10}$ length with frequency is -0.57 and the Spearman correlation -0.51. The stage `inventory` ran in 8.6 s and `significance` in 1.2 s.

4.11.4 Issues found and corrections

The first status (Section 4.3) had no degraded sub-link, because its only degraded rule never fired. It also labelled `sublink_1` of links 10034 and 10074 as good, because it used RSL completeness, while their TSL, and hence TL, was mostly missing. The new status uses TL completeness and further rules; the old status is kept in the column `status_step3`, and the frozen split was not changed.

4.12 Per-link reference state

4.12.1 Purpose

Network events are coarse for a single link: an event that is wet somewhere in the domain may be dry over a given link. The per-link reference state labels every link and 5 min bin from the references only, so that CML data can be labelled for training without the labels depending on the CML itself.

4.12.2 Method

The stage `link_state` (`src/cmlpipe/events/link_state.py`, `classify_links`) applies the following rules (keys under `link_state`):

- A bin is wet when `R_max` along the path (thresholded at 0 dBZ) is at least 0.1 mm/h, or a gauge within 1 km of the path has at least 0.1 mm in the bin.
- A bin is dry when it is not wet, `R_max` is 0 in the bin and in every bin within ±30 min, and every gauge within 1 km is 0 and not missing over the same window; a link without

such a gauge passes the gauge condition.

- A bin is unknown otherwise: radar missing in the window, light rain or rain in the margin, or a gauge missing.

The state is stored as int8 (0 dry, 1 wet, −1 unknown) together with a reason bitmask (1 radar wet, 2 gauge wet, 4 radar missing in the window, 8 rain in the margin) in `link_reference_state_5min.nc`. City gauges are used at 5 min; the 15 min SMHI amount labels each of its three 5 min bins, which is a state label and not a disaggregated amount. At the period edges the window is truncated (`window_any`).

4.12.3 Verification and results

Tests check the truncation at the edges, the rules and that a missing gauge prevents the dry state. Over JJA 77.1 % of the link bins are dry, 8.8 % wet and 14.1 % unknown; inside significant events 32.3 % dry, 30.3 % wet and 37.4 % unknown; outside events 96.6 % dry, 0.0 % wet and 3.4 % unknown. When the network state is dry, 97.0 % of the link bins are dry; when it is wet, 31.0 % are wet, 31.1 % dry and 37.9 % unknown. The reasons occur in 8.6 % (radar wet), 0.9 % (gauge wet), 2.3 % (radar missing) and 12.4 % (margin) of the link bins.

4.12.4 Issues found and corrections

No issue required a correction.

4.13 Figures and the run report

4.13.1 Purpose

Many data problems are recognised only in figures. Four stages draw fixed sets of figures from the stored products and collect all numbers in one report.

4.13.2 Method

The stage `inventory_figures` (`src/cmlpipe/viz/inventory_figures.py`) draws the daily TL availability per link ordered by length and by completeness, length against frequency with the split, the distributions of completeness, worst month, dry noise, baseline range and TSL change rate with the status limits, a flag gallery with one 10 s example per flag chosen by fixed rules (the spike example is the isolated spike with the largest jump relative to the dry noise on a quiet usable link, shown ±5 min), and network maps by status, split and completeness.

The stage `quicklooks` (`quicklooks.py`) draws one figure per significant event and selected link, with four panels: `tl_mean` of both sub-links with blocking flags marked, radar along the path, the rate of the nearest gauge, and a timeline of the QC flags. The window is the event padded by 12 h on both sides; padding is shaded grey and other events in the padding are marked

with their identifiers. A descriptive overview per event shows the largest rise of `tl_mean` over the 3 h pre-event median versus path length, and the share of sub-links more than 3 dB above their pre-event median in each 5 min bin. The 12 links are chosen by a fixed rule (keys under `quicklooks`): links not excluded from the split; within each cell of length class and frequency band, preference for the test split, then both sub-links good, then a gauge within 1 km, then the nearest gauge, then `cml_id`; two links per cell, then filled round-robin.

The stage `link_cards` (`link_cards.py`) draws one page per link with the metadata (length, frequencies, polarisation, sites, split, status of both sub-links, gauges within 1 km, nearest gauge, TSL behaviour), 92-day strips of hourly `tl_mean` with network-outage bands and blocking-flag marks, `rsl_mean` and `tsl_mean`, the dry-period medians of TL, and an inset map of the path on the radar grid with the crossed cells shaded by weight; an index page lists all cards. No combined PDF is produced, because rendering the figures twice would double the run time. The stage `run_report` (`run_report.py`) writes one self-contained HTML page with the sections run information, data coverage, QC summary, geometry, aggregation, events, inventory, quicklooks and link cards, and known limitations; every number is read from the stored tables, the manifest and the file attributes, and the limitations are generated from those facts. The figure stages run on 4 workers and use a shared style (`style.py`).

4.13.3 Verification and results

The acceptance confirmed 420 quicklooks (35 significant events × 12 links), 35 overviews, 364 link cards and a run report of 13 kB with 9 sections. A test checks the quicklook selection rule. The run times were 134.3 s for `quicklooks`, 125.5 s for `link_cards`, 0.4 s for `run_report` and 1.9 s for `event_links_extra`.

4.13.4 Issues found and corrections

After the nearest gauge at any distance was added to the link table, one quicklook link changed (10153 to 10238), because the tie-break of the selection now used real distances. Legends for event bounds, wet core and padding were added to the quicklooks after the first review.

4.14 Release

4.14.1 Purpose

The research code needs one fixed, documented dataset that cannot change under it, in formats that allow fast access to any link and period without loading whole files.

4.14.2 Method

The stage `release` (`src/cmlpipe/release/build.py`, `run_release`) builds `data/release/openmrg_v1/` in three steps (Figure 7). First it writes everything into `openmrg_v1.building/`.

`write_tables` writes the tables as Parquet and CSV with consistent keys (`normalise`: sub-links as `<cml_id>_<1|2>` with an integer column `sublink`, `gauge_id` instead of `id`); for the frozen tables the CSV is a byte copy of the work file. `write_timeseries` writes the Zarr stores (`cmlstores.py`, `zarrstore.py`): the 10 s store holds TSL, RSL and TL as exact copies of L1 and the L2 flags, the other stores the L3 variables, written one variable at a time over the whole period without `dask`. The chunk shapes were chosen by an archived benchmark for the common access "one link over a long period"; it compared, for example, one day × all sub-links with one week × 64 sub-links at 10 s. The file `catalog.json` describes every product, variable, unit, time span and stage hash and the sha256 of every table; `DATA_DICTIONARY.md` and `README.md` are generated from the same metadata (`metadata.py`), so that the descriptions cannot disagree; `DATA_GUIDE.md` (`guide.py`) is a plain-language page for a reader without knowledge of the pipeline, including how to obtain the data (rebuild from the Zenodo download, or download a release archive and check its SHA256); `LICENSE.txt` states the licence of the data.

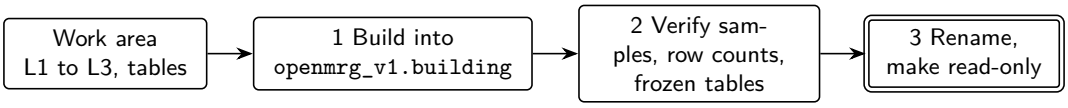


Figure 7: The three steps of the release stage. Nothing in the work area changes.

Second, `verify` compares 20 random combinations of sub-link, day and resolution between the Zarr stores and the work NetCDF files (integers exactly, floating-point values to within 10^{-6}), compares the row counts of all tables, and requires the frozen tables to be byte-identical; the result is `verification.json`. Third, the folder is renamed to `openmrg_v1` and made read-only. An existing release is never overwritten without `--force`; `--version v2` creates a new version beside it. The command `python -m cmlpipe guide` rewrites only the text files of a release. The contents of the release are listed in Section 7.

4.14.3 Verification and results

For `openmrg_v1` all 20 samples were equal, the row counts matched and `frozen_unchanged` is true. Tests check the key normalisation, that every table column is described in the metadata, and that the stage refuses to overwrite a release without `--force`. The release occupies 1.6 GB; the work area 2.3 GB.

4.14.4 Issues found and corrections

The release was first built on 2 October 2026 between 18:08 and 18:22 and rebuilt at 20:41 with complete variable descriptions. Before the code was reorganised (Section 6.4), the key results were saved (`_archive/baseline_pre_step8.json`) and compared afterwards to confirm that the reorganisation changed no result.

5 Verification

5.1 Approach

Verification combined four means. Acceptance reports compared the outputs with independent values, mostly from the dataset paper [1], under tolerances written down before the run; where no tolerance was given, half the resolution of the data was used. Independent check scripts read the raw files by a route different from the pipeline code (`check_step1.py`, `check_step2.py`, `check_step2b.py`, `check_step5.py` and `check_step8.py`, archived in `_archive/checks/`). Visual pilots inspected examples of every flag and every figure type. An automated test suite runs on small synthetic datasets. The results of each stage are given in its subsection of Section 4; this section holds the comparison of the reference data with the published values, which concerns several stages, and the index of the test suite.

5.2 Acceptance of the reference data

Table 39 lists the 39 checks of the L1 reference data against the dataset paper; all passed. Gauge totals are computed on interval-start labels and UTC days. The tolerances of the individual gauge totals equal half the rounding of the published totals. Three further rows of the acceptance report were information only: the SMHI quality codes (Section 3.3), the radar completeness per cell (Section 3.4) and the Torslanda window (Section 3.5), which is 14:29 to 17:29 in L1 labels.

Table 39: Acceptance of the L1 reference data (reference values from [1] unless stated).

Quantity	Pipeline	Reference	Tolerance
SMHI GbgA total over JJA	255.9 mm	256.0 mm	±1 mm
SMHI GbgA wet fraction, 15 min (≥ 0.1 mm)	5.53 %	5.50 %	±0.3 %
SMHI GbgA largest 15 min amount (29 July 2015 07:45)	7.1 mm	7.1 mm	±0.05 mm
SMHI GbgA largest daily total (17 June 2015)	25.5 mm	25.5 mm	±0.05 mm
Mean JJA total, 7 City weighing gauges	263.0 mm	263.0 mm	±3 %
Mean JJA total, 3 City tipping-bucket gauges	219.2 mm	219.0 mm	±3 %
Jarn total	236.6 mm	237 mm	±0.5 mm
Torp total	271.2 mm	271 mm	±0.5 mm
Bergsj total	301.1 mm	301 mm	±0.5 mm
Torsl total	272.8 mm	273 mm	±0.5 mm
Chalm total	277.2 mm	277 mm	±0.5 mm
Tole total	250.8 mm	251 mm	±0.5 mm

Quantity	Pipeline	Reference	Tolerance
Barl total (2880 missing steps)	231.5 mm	232 mm	±0.5 mm
Drakeg total	185.4 mm	185 mm	±0.5 mm
Lbom total	227.6 mm	228 mm	±0.5 mm
Askim total	244.6 mm	245 mm	±0.5 mm
Radar 5 min steps	26 496	26 496	exact
Radar first stamp	06-01 00:05	raw file	exact
Radar last stamp	09-01 00:00	raw file	exact
Radar raw stamps missing in L1	none	none	exact
Radar L1 stamps not in the raw file	none	none	exact
Radar dBZ in L1 against raw (NaN pattern identical)	$1.9 \cdot 10^{-6}$ dBZ	0	10^{-4}
Radar steps without any valid cell	115	115	exact
Radar steps with at least one valid cell	99.57 %	99.60 %	±0.2 %
Largest reflectivity	58.8 dBZ	58.8 dBZ	±0.2 dBZ
Radar grid (y × x)	48 × 37	48 × 37	exact
Weather first and last stamp	06-01 01:00, 09-01 00:00	raw file	exact
Weather raw stamps missing in L1	none	none	exact
Weather L1 stamps not in the raw file	none	none	exact
Temperature: valid values, equal to raw	2208	2208	exact, 10^{-4}
Humidity: valid values, equal to raw	2106	2106	exact, 10^{-4}
Pressure: valid values, equal to raw	2208	2208	exact, 10^{-4}
Wind speed: valid values, equal to raw	2208	2208	exact, 10^{-4}
Wind direction: valid values, equal to raw	2208	2208	exact, 10^{-4}
Gust: valid values, equal to raw	2208	2208	exact, 10^{-4}
Torslanda: Torsl total in the window	18.5 mm	18.5 mm	±0.1 mm
Torslanda: largest 1 min amount	1.1 mm	1.1 mm	±0.05 mm
Torslanda: peak minute (interval start)	15:04	15:04	exact
Torslanda: L1 Torsl equal to the folder series	181 of 181	all	exact

5.3 Automated tests

The test suite is in `tests/` and is run with `python -m pytest` from `pipeline/`. The synthetic tests use small datasets built in `conftest.py` (Section 4.1); during the tests the environment variable `HDF5_PLUGIN_PATH` is set to avoid a slow search for compression plug-ins. Tests that need the real data or the work and release products carry the marker `data` and are excluded by default; `python -m pytest -m data` runs only them. On 7 October 2026 the default run gave 128 passed tests in about 50 s, with 15 tests deselected. Table 40 indexes the test files by the section that describes their tests; parametrised tests count once.

Table 40: Test files.

File	Tests	Marker	Section
`test_openmrg_adapter.py`	11		4.1
`test_schema.py`	11		4.1
`test_real_openmrg.py`	3	`data`	4.1
`test_reference_ingest.py`	10		4.2
`test_stage_hash.py`	5		5.3
`test_qc.py`	13		4.3
`test_geometry.py`	21		4.4
`test_aggregate.py`	21	one with `data`	4.5, 4.6
`test_events.py`	18		4.7, 4.8
`test_inventory.py`	9		4.10 to 4.13
`test_release_linkstate.py`	6	one with `data`	4.12, 4.14
`test_api.py`	10	`data`	5.3
`test_api_event_pairs.py`	1		4.9
`test_boundaries.py`	2		6.2

Two groups of tests are not described elsewhere. The per-stage hash tests confirm that every stage has a hash, that changing an unrelated key leaves the other stages unchanged, that changing a used key changes its stage, that the output folder is not part of the hash, and that a missing key raises an error. The real-data tests of `CMLData` confirm that the tables equal the work tables, that link and event filters work, that the L3 and 10 s CML data, radar, gauges, weather and states equal the work files, that `frame` is aligned, that the event markers are correct, that `describe` and the errors work, that times are UTC with interval-start labels, and the event pairs.

6 Software organisation

6.1 Structure

The code is a Python package, `cmlpipe` (version 0.2.0), in `src/cmlpipe/`, installed from `pyproject.toml` (Python 3.11 or later); its dependencies are listed in [4] and their tested versions in Table 51. Each processing package is split into a `core.py` module with pure computations without file access, which are tested in isolation, and a `stage.py` module that reads and writes files and calls them. Tables 41 to 47 list the modules by package; the functions are described in the sections indicated.

Table 41: Package `cmlpipe`: infrastructure (Section 2).

Module	Responsibility	Principal functions
`cli.py`, `__main__.py`	Command line, stage order	`main`, `build_parser`, `STAGES`, `DAY_STAGES`
`config.py`	Configuration, paths, stage hash	`load_config`, `Config.stage_hash`, `config_hash`
`provenance.py`	Output attributes, sidecars, manifest	`git_commit`, `provenance_attrs`, `write_sidecar`, `append_to_manifest`
`parallel.py`	Day stages on several processes	`run_day_task`, `run_days`
`logging_utils.py`	Console and file logging	`new_run_id`, `setup_logging`

Table 42: Package `cmlpipe.io` (Sections 4.1 and 4.2).

Module	Responsibility	Principal functions
`openmrg.py`	Raw CML file and metadata to L1	`load_cml_metadata`, `load_cml_day`, `write_l1_day`, `check_roundtrip`
`openmrg_reference.py`	Gauges, weather, radar to L1	`load_city_gauges`, `load_smhi_gauge`, `load_smhi_weather`, `load_radar_month`
`schema.py`	L1, L2 and L3 formats and validators	`validate_cml_l1`, `validate_radar_l1`, `validate_l3`, `zr_rain_rate`
`ingest.py`	The four ingest stages	`ingest_cml`, `ingest_gauges`, `ingest_met`, `ingest_radar`
`netcdf.py`	NetCDF attribute and core-file reading	`existing_stage_hash`, `read_core_file`

Table 43: Package `cmlpipe.qc` (Section 4.3).

Module	Responsibility	Principal functions
`flags.py`	The eight flag bits	`QCFlag`, `mask_of`, `decode_flags`
`rules.py`	Flag rules	`flag_rx_floor`, `flag_spike`, `flag_flatline`, `flag_network_gap`, `combine`
`stage.py`	Quality-control stages	`floor_level`, `compute_flags`, `write_l2`, `sublink_status`

Table 44: Packages `cmlpipe.geometry` and `cmlpipe.aggregate` (Sections 4.4 to 4.6).

Module	Responsibility	Principal functions
`geometry/core.py`	Projections, grid, weights, distances	`RadarGrid`, `path_grid_weights`, `point_to_path`, `merge_sites`, `apply_weights`
`geometry/stage.py`	Stage `geometry`	`run_geometry`, `read_weights`
`aggregate/core.py`	Vectorised reductions	`bin_statistics`, `aggregate_cml_day`, `aggregate_gauges`, `floor_rain`
`aggregate/stage.py`	Aggregation stages	`write_verified`, `run_aggregate_radar`

Table 45: Packages `cmlpipe.events` and `cmlpipe.inventory` (Sections 4.7 to 4.12).

Module	Responsibility	Principal functions
`events/core.py`	Event logic and split	`classify`, `find_events`, `certified_dry`, `split_events`, `split_links`
`events/stage.py`	Stages `events`, `split`, `event_links`	`build_reference`, `build_core`, `event_table`, `event_links_table`
`events/link_state.py`	Stage `link_state`	`classify_links`, `window_any`
`inventory/core.py`	Significance and status rules	`significance`, `event_runs`, `tsl_behaviour`, `quality_status`
`inventory/stage.py`	Stages `significance`, `inventory`, `event_links_extra`	`build_sublinks`, `build_links`, `build_gauges`

Table 46: Packages `cmlpipe.release`, `cmlpipe.api` and `cmlpipe.viz` (Sections 4.13, 4.14 and 7).

Module	Responsibility	Principal functions
`release/build.py`	Stage `release`	`write_tables`, `write_timeseries`, `verify`, `run_release`
`release/cmlstores.py`, `zarrstore.py`	Zarr writing	`write_cml_10s`, `write_cml_res`, `open_store`
`release/metadata.py`	Single source of all descriptions	`column_description`, `variable_description`
`release/guide.py`	Text files of a release	`data_guide`, `write_guide`
`api/cmldata.py`	Reading the release	`CMLData`
`api/access.py`, `explore.py`	Earlier helpers on the work folder	`load_link_window`, `find_links`, `show_event`
`viz/`	Report figures and explorer figures	`make_event_quicklook`, `make_link_card`, `link_view_figure`

Table 47: Applications and tools outside the package.

File	Responsibility	Section
`apps/explorer/app.py`, `data.py`, `smoke.py`	Explorer	7.3
`start_explorer.bat`	Explorer launcher; uses the miniconda Python if present	7.3
`tools/check_boundaries.py`	Boundary check	6.2
`tools/update_readme.py`, `tools/githooks/pre-commit`	Generated README sections	6.3
`examples/quickstart.ipynb`	Introductory notebook	7.2

6.2 Boundary between data layer and research

The dependency between the two parts goes in one direction. A file under `research/` may import from `cmlpipe` only `cmlpipe.api` and `cmlpipe.viz`, and no file under `pipeline/` may import research code. The research code reads only the release pinned in `research/config.yaml` (currently `openmrg_v1`). The script `tools/check_boundaries.py` reads the imports of all files with the Python syntax tree, without running them, and exits with an error on a violation; a test confirms that the repository respects the boundary and that the checker detects violations.

6.3 Generated documentation

Each README file has hand-written text and generated blocks between the markers `<!- AUTO:name ->` and `<!- /AUTO:name ->`. The script `tools/update_readme.py` replaces only those blocks: the root README receives one status line per part from the two roadmaps, `pipeline/README.md` the processed data on the computer and the progress from `pipeline/docs/roadmap.md`, and `research/README.md` the progress from `research/ROADMAP.md`. Progress is edited by hand only in the two roadmaps. The script uses only the Python standard library and reads facts about the code by parsing the source files, not by importing them. It runs on every commit through the git hook (enabled with `git config core.hooksPath pipeline/tools/githooks`); the hook locates the miniconda Python through `USERPROFILE`.

The documentation of the data layer consists of `docs/methods.md` (final definitions of all products), `docs/data_dictionary.md` and `docs/DATA_GUIDE.md` (generated with the release), `docs/howto_access.md`, `docs/roadmap.md`, `docs/openmrg_summary.md` (summary of the dataset), `docs/papers/` (the dataset paper) and `docs/reference/` (notes on the OpenSense tools and an example notebook of the OpenSense community).

6.4 Reorganisation

After seven development steps the code was reorganised. Every file under version control (83 files) and every data folder (63 folders) was listed and classified. The pipeline code moved from flat modules into the package layout of this section; acceptance reports, calibration notes, check and inspection scripts and report generators moved to `_archive/` (outside version control); the data moved from `data_processed/` to `data/work/`; one command line replaced the earlier script; the flag bits were defined once in `QCFlag`; and dataset paths were removed from the stage hashes. Later the data layer was moved under `pipeline/`; because no path is hashed, the full run afterwards reported every stage as up to date. The dates and the renamed files are given in Section 9.

7 Access to the data

7.1 Release contents

The release `data/release/openmrg_v1/` (Section 4.14) contains the folders `tables/` and `timeseries/` and the files `catalog.json`, `DATA_DICTIONARY.md`, `DATA_GUIDE.md`, `README.md`, `LICENSE.txt` and `verification.json`. Table 48 lists the 16 tables, each provided as Parquet and CSV, and Table 49 the 20 time-series stores. The time and unit conventions of Sections 3.6 and 3.7 apply to all products, and no baseline has been applied.

Table 48: Tables of the release.

Table	Rows	Section
`links`, `sublinks`, `gauges`	364, 728, 11	4.11
`sites`, `link_gauge_matches`	418, 852	4.4
`events`, `dry_periods` (frozen)	89, 62	4.7
`sublink_dry_stats`	728	4.7
`split_events`, `split_links` (frozen)	89, 364	4.8
`event_links` (frozen), `event_links_flags_extra`	32 396 each	4.9
`event_significance`, `event_runs`	89, 368	4.10
`quality_status`	728	4.11
`network_gaps`	93	4.3

Table 49: Time-series stores of the release.

Store	Dimensions	Chunks	Variables
`cml_10s`	794 880 × 728 sub-links	60 480 × 64	`tsl`, `rsl`, `tl`, `flags`
`cml_1min`	132 480 × 728	132 480 × 32	Table 23, counts, `flags_or`, `n_tsl_step`
`cml_5min`	26 496 × 728	8928 × 64	as `cml_1min`
`cml_15min`	8832 × 728	8832 × 64	as `cml_1min`
`cml_1h`	2208 × 728	2208 × 728	as `cml_1min`
`radar_along_links_5min`, `_15min`, `_1h`	time × 364 links	8928 × 64 at 5 min	Section 4.6

Store	Dimensions	Chunks	Variables
`radar_at_gauges_5min`, `_15min`, `_1h`	time × 11 gauges	one per variable	`R_pixel`, `R_3x3` and valid counts
`gauges_city_1min` to `_1h` (4 stores)	time × 10 gauges	one per variable	`amount_mm`, `rate_mm_h`, `n_valid`, `peak_1min_rate_mm_h`
`gauges_smhi_15min`, `_1h`	time × 1 gauge	one per variable	`amount_mm`, `rate_mm_h`, `n_valid`, `qc_worst`
`met_1h`	2208 × 1	one per variable	Weather variables and quality codes
`network_reference_5min`	26 496	one per variable	Section 4.7
`link_reference_state_5min`	26 496 × 364	8928 × 64	Section 4.12

7.2 Programming interface

The class `CMLData` (`src/cmlpipe/api/cmldata.py`) opens a release with `CMLData.open`, which reads only `catalog.json`; tables and stores are opened on first use and only once per object. Tables are returned as pandas DataFrames and time series as lazy xarray datasets, so that only the requested chunks are read. Arguments `start` and `end` accept text or date values (values with a time zone are converted to UTC) and select [start, end); identifiers may be link numbers or sub-link identifiers. A bad request (unknown identifier or resolution, empty or too large window) raises `CMLDataError`. Table 50 lists the methods.

The method `cml` applies quality control to the 10 s data in three modes: `mask` sets RSL, TSL and TL to NaN where a blocking flag of the respective set (Table 11) is present; `raw` returns the values as recorded; `flags` returns the raw values with the flag bitmask. At 1 min and coarser the statistics already exclude blocked samples, and `flags` adds `flags_or`. A 10 s request longer than 7 days, or for more than 50 sub-links over more than 2 days, requires `allow_large=True`. The following lines show typical use.

```
from cmlpipe.api import CMLData
d = CMLData.open("data/release/openmrg_v1")
links = d.links(length_km=(1, 3), frequency_band=">=25GHz", status="good",
                split="train", has_gauge_within_m=1000)
ev = d.events(significant=True, split="train")
ds = d.cml(links.cml_id[:5], "2015-07-28", "2015-07-29", "5min")
df = d.frame(10227, "2015-07-28 12:00", "2015-07-28 20:00", res="5min")
w = d.event_window("E20150727_2140", cml_id=10227, pad="3h")
pairs, dropped = d.event_pairs(split=("test", "train"), return_dropped=True)
```

Table 50: Methods of `CMLData`.

Method	Returns
`table`, `store`	Any release table or time-series store by name
`links`, `sublinks`	Links or sub-links, filtered by length, frequency band, status, split, gauge distance
`gauges`	Gauges
`events`	Events joined with their significance, filtered by significance, split, month
`event_pairs`	Event pairs for modelling (Section 4.9)
`event_runs`, `dry_periods`	Wet runs of an event; dry periods
`status`	Quality status per sub-link with the metrics behind it
`cml`	CML time series, dimensions time and `sublink_id`, with metadata as coordinates
`radar_along`, `gauge`, `met`	Radar along paths, gauges, weather
`link_state`, `network_state`	Per-link and network reference series
`frame`	One aligned table for one link on one time axis
`event_window`	`frame` of an event with padding and markers for event, wet run and dry period
`describe`	Where a variable appears, with unit and description

```
d.describe("tl_mean")
```

The method `frame` returns both sub-links (columns ending in `_s1` and `_s2`), the flags, the radar along the path, the gauges, the weather and the reference state. The earlier helpers `load_link_window` (`api/access.py`) and `find_links`, `find_events`, `show_link` and `show_event` (`api/explore.py`) read the work folder and remain available. The notebook `examples/quickstart.ipynb` and `docs/howto_access.md` describe the use further.

7.3 Explorer

The explorer is a local web application built with Streamlit [6], started with `python -m cmlpipe explore` or `start_explorer.bat`. It serves the address `http://localhost:8501`, reachable only from the same computer. It has three layers (Figure 8); the figure builders have no file access, so that the same figures can be produced in a notebook. Results are cached, and every selection is kept in the session state, so that it survives reruns and changes of tab.

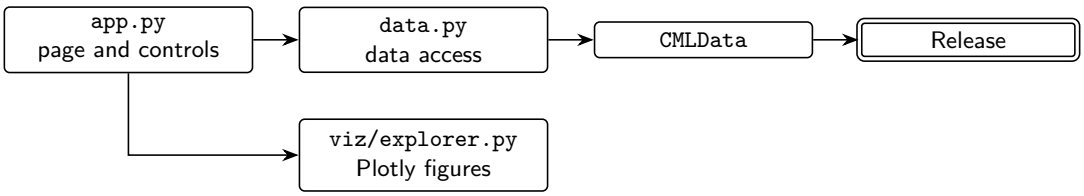


Figure 8: Layers of the explorer. Arrows point to what a layer calls.

The side panel offers, from top to bottom: link filters (length, frequency band, polarisation, status, split, gauge within a distance, minimum TL completeness); a link selector with previous, next and random buttons; the time window, either an event with 0 to 48 h of padding (events

filtered by significance, split or month) or a free window with shift buttons (±1 h, ±6 h, ±1 day), whole month and whole period; the resolution (10 s to 1 h, with 10 s limited to 3 days and an offer to switch to 1 min); the layers (TL, RSL and TSL; radar `R_mean`, `R_max` and wet fraction; gauge rate and cumulative amount; weather; QC flags; reference-state shading; event bounds and wet runs; network gaps; dry baseline; minimum and maximum band and standard deviation of TL); the gauges shown (nearest City, nearest SMHI, both, or all within a radius); display options (blocked samples as NaN or raw, lines or markers, equal dB span on the signal panels, TL relative to the dry baseline, logarithmic rain axis, a palette safe for colour-vision deficiency); and an advanced option to add any numeric column.

The explorer has six tabs. The link view is one figure with stacked panels on a shared time axis, so that zooming one panel zooms all; long traces are reduced to 20 000 points per trace by keeping the minimum and maximum of equal buckets (`minmax_decimate`), so that peaks survive, and a note states when this happened; exports are never reduced. Radar and gauges coarser than the chosen resolution are drawn as steps and never interpolated. The map shows all links coloured by status, split or completeness, with the selected link and its nearest gauges; it requires an internet connection for the map tiles. The events tab shows a table of all events, in which a click opens the event in the link view, and two descriptive figures over all links (rise of TL above the dry median against length, and the share of links at least 1 dB above their dry median over time). The link information tab shows the metadata, the metrics of both sub-links, an hourly TL plot over the summer with dry periods marked, and the gauges within 1 km. The network tab shows a heatmap of the valid TL share per link and day, length against frequency, and histograms. The export tab writes the displayed data as CSV or Parquet and gives a Python snippet that reproduces them with `CMLData`.

The dashed "dry median" line is the median TL over the dry periods. It is a descriptive reference and not the baseline estimation of the proposal [8]; the TL rise of the events tab is descriptive as well and contains no wet-antenna correction. The script `apps/explorer/smoke.py` builds every figure without a browser for five scenarios, with timings: the event `E20150727_2140` on link 10316 at 5 min and at 10 s, a flatline example, an unusable link and a whole-summer link.

7.4 Installation

The environment is created with `conda env create -f pipeline/environment.yml`, which also installs the package in editable mode, or with `pip install -e "pipeline[dev,explorer]"`. The file `environment.yml` records the versions tested on 2 October 2026 under Windows 10 (Table 51). The project uses the miniconda Python of the computer; the Anaconda installation lacks `netCDF4` and has an older NumPy.

Table 51: Tested library versions.

Library	Version	Library	Version
Python	3.12	Matplotlib	3.10.9
NumPy	2.4.5	PyArrow	25.0.1
pandas	3.0.6	Zarr	3.4.0 (format 2 stores)
xarray	2026.9.0	Plotly	7.1.0
netCDF4	1.7.4	Streamlit	1.65.0
pyproj	3.8.0	poligrain	0.3.1
Shapely	2.1.2		

8 Limitations and open work

8.1 Limitations of the data

The data and the processing impose the following limitations on later work; the measurements behind each item are given in the section indicated.

- The radar is adjusted to gauges and is therefore not an independent reference; it reads less than the gauges at most gauge cells and missed the first peak of the Torslanda event (Section 4.6).
- The map datum of the radar grid is not documented (Section 3.4); a datum error moves part of a path into a neighbouring cell (Section 4.4).
- The radar time stamp was assigned by an empirical lag test with a resolution of one radar step (Section 3.6).
- The tipping-bucket City gauges collect less rain than the weighing gauges, which is probably an undercatch (Table 39), and the height of the City gauges is an assumption (Section 3.3).
- Few links have a gauge near the path, and many short links lie inside a single radar cell, where the radar cannot resolve rain along the path (Section 4.4).
- Link length and frequency are correlated, so their effects on attenuation cannot be separated without care (Section 4.11).
- Most events are drizzle only, and most events are merged from several wet runs, so network event bounds are coarse for a single link; the wet runs or the per-link state are more precise (Sections 4.7 to 4.12).
- The status rules based on network percentiles mark part of the sub-links by construction (Section 4.11).
- Some links change TSL during rain, and some toggle TSL continuously, although transmit power control is documented as disabled; attenuation should be computed from TL on those links (Sections 4.3 and 4.11).
- The link split is not a spatial hold-out, since neighbouring links observe the same rain and share sites (Section 4.8).

- The quality-control thresholds were calibrated on this dataset and are not validated on other networks (Section 4.3).

8.2 Open work

The following items of the pipeline roadmap (`pipeline/docs/roadmap.md`) are open: wet and dry classification from the CML signal itself; noise and outlier handling beyond the flags (level shifts, mismatch between the two directions of a link); link selection for the two research stages; an adapter for OpenRainER, whose sub-links are named `channel1` and `channel2`; and a check whether OpenMesh provides a rain reference. Baseline estimation, from the dry periods and by the minimum-attenuation method [8], and the computation of $A(t)$ in Eq. (1) are the first research tasks (`research/ROADMAP.md`); they must ensure that the baseline does not follow the drying tail. They are followed by the wet-antenna model, the path law $f(L, \bar{R}, Z)$ and rain estimation. None of the research tasks has been started. Small open items in the documentation are the placeholders for the supervisor and the dates in the root README and in `research/README.md`.

9 Chronology

The data layer was developed in eight steps in three sessions between 30 September and 2 October 2026 and reorganised on 2 and 4 October 2026 (Table 52). The commits before 2 October 2026, 20:51, are kept on the branch `dev-history`, with the tag `pre-step8` at the state before the reorganisation; the branch `main` starts with one commit, tagged `data-openmrg-v1`. Table 53 lists the files that were renamed during the reorganisation (Section 6.4).

Table 52: Chronology (Israel time, UTC+3).

Date	Time	Work	Commit
2026-09-30	14:12	Step 1: first ingest of one day (2015-07-28)	none
2026-09-30	14:39	Ingest of all 92 CML days	none
2026-09-30	15:14	First commit: documents, adapter, schema, configuration, provenance, tests	`851f30e`
2026-09-30	15:22	README generated from the roadmap; pre-commit hook; hook corrected to find miniconda	`bfe109f`, `d65ea5a`
2026-09-30	15:39	Step 2: ingest of gauges, weather and radar	none
2026-09-30	16:40	Step 2b: radar and weather rebuilt after the period-window correction	none
2026-09-30	16:42	Steps 2 and 2b committed with the acceptance report	`c1580b4`
2026-09-30	16:49	Per-stage hashes; check scripts moved	`4bb51a6`

Date	Time	Work	Commit
2026-09-30	16:50 to 16:57	All L1 files rebuilt with stage hashes	`b7415ef`
2026-09-30	17:00 to 17:24	Step 3: QC statistics, first pilot, recalibration, second pilot	none
2026-09-30	17:31	Step 3 committed: QC flags	`859c7ed`
2026-09-30	17:39	QC flags for all 92 days	none
2026-09-30	18:37	First status table	none
2026-10-01	23:37 to 23:43	Step 4: geometry	`0bd3cb5`
2026-10-01	23:54 to 23:59	Independent geometry check added and removed; outputs regenerated	`5b2ef6f`, `5ebf2bf`, `e4c3c2c`
2026-10-02	00:06 to 00:22	Step 5: aggregation pilot; parallel workers	`ca04961`
2026-10-02	00:35	Aggregation of all days with 4 workers	none
2026-10-02	15:53 to 16:07	Step 6: events, dry periods, split, event pairs	`6a4b111`
2026-10-02	16:40 to 17:01	Step 7: significance, inventory, status, figures, run report	`9072e9d`
2026-10-02	17:29	Step 7b: figure revisions, nearest gauge, flag extras	`6e9bb3c`
2026-10-02	17:51 to 17:54	Step 8: package layout, single command line, link state	`d77a190`
2026-10-02	18:08 to 18:37	Release builder, `CMLData`, methods documentation, notebook	`211f942`
2026-10-02	20:41 to 20:51	Release rebuilt; full run; explorer; clean history on `main`	`62e56c9`, `75f6a31`
2026-10-04	14:20 to 15:43	Tests and explorer revised; release guide rewritten	none
2026-10-04	18:07 to 18:14	Data layer moved under `pipeline/`; full run, all stages up to date	`f240609`
2026-10-04	19:16	One README per part; roadmap split into pipeline and research	`8416f85`

References

[1] J. C. M. Andersson, J. Olsson, R. (C. Z.) van de Beek and J. Hansryd, “OpenMRG: Open data from Microwave links, Radar, and Gauges for rainfall quantification in Gothenburg,

Table 53: Renamed files (paths before 2 October 2026 and on 7 October 2026).

Before	Now
`pipeline/run.py`	`pipeline/src/cmlpipe/cli.py`
`pipeline/config/openmrg.yaml`	`pipeline/configs/openmrg.yaml`
`pipeline/cmlpipe/adapters/openmrg.py`	`pipeline/src/cmlpipe/io/openmrg.py`
`pipeline/cmlpipe/adapters/openmrg_reference.py`	`pipeline/src/cmlpipe/io/openmrg_reference.py`
`pipeline/cmlpipe/schema.py`	`pipeline/src/cmlpipe/io/schema.py`
`pipeline/cmlpipe/qc.py`, `qc_runner.py`	`pipeline/src/cmlpipe/qc/rules.py`, `stage.py`
`pipeline/cmlpipe/geometry.py`, `geometry_runner.py`	`pipeline/src/cmlpipe/geometry/core.py`, `stage.py`
`pipeline/cmlpipe/aggregate.py`, `aggregate_runner.py`	`pipeline/src/cmlpipe/aggregate/core.py`, `stage.py`
`pipeline/cmlpipe/events.py`, `events_runner.py`	`pipeline/src/cmlpipe/events/core.py`, `stage.py`
`pipeline/cmlpipe/inventory.py`, `inventory_runner.py`	`pipeline/src/cmlpipe/inventory/core.py`, `stage.py`
`pipeline/cmlpipe/quicklooks.py`, `link_cards.py`, `run_report.py`	`pipeline/src/cmlpipe/viz/`
`pipeline/cmlpipe/plotstyle.py`	`pipeline/src/cmlpipe/viz/style.py`
`pipeline/cmlpipe/access.py`, `explore.py`	`pipeline/src/cmlpipe/api/`
`pipeline/tests/test_acceptance_openmrg.py`	`pipeline/tests/test_reference_ingest.py`
`.githooks/pre-commit`	`pipeline/tools/githooks/pre-commit`
`data_processed/openmrg/`	`data/work/openmrg/`

Sweden," *Earth System Science Data*, vol. 14, pp. 5411–5426, 2022. doi:10.5194/essd-14-5411-2022.

[2] OpenMRG dataset, version 1.1, Zenodo, doi:10.5281/zenodo.7107689 (CC BY-SA 4.0). The file itself names doi:10.5281/zenodo.6673750; the relation between the two identifiers was not verified.

[3] M. Fencl et al., OpenSense data-format conventions for opportunistic rainfall sensors, *Open Research Europe*, vol. 3, art. 169, 2023. https://open-research-europe.ec.europa.eu/articles/3-169 (full title and author list not verified).

[4] NumPy, pandas, xarray, netCDF4, PyYAML, pyproj, Shapely, Matplotlib, PyArrow, Zarr and Plotly: open-source Python libraries for computation, file formats, map projections, geometry and figures; optional pytest and Streamlit (dependencies listed in `pipeline/pyproject.toml`).

[5] `poligrain`, version 0.3.1, Python library of the OpenSense community.

[6] Streamlit, open-source Python framework for data applications.

[7] Plotly, open-source Python plotting library.

[8] J. Ostrometzky and H. Messer, "Dynamic determination of the baseline level in microwave links for rain monitoring from minimum attenuation values," *IEEE Journal of Selected*

Topics in Applied Earth Observations and Remote Sensing, 2018.

A Glossary

Table 54 indexes the defined terms by the section in which each is defined.

Table 54: Index of defined terms.

Term	Section	Term	Section
Alive sub-link	4.3	JJA	3
Blocking flag	4.3	Level (raw, L1, L2, L3)	2
Consolidated core file	4.7	Link, site, sub-link	3.2
Day stage	2.2	Matched gauge	4.9
Domain	4.7	Network reference series	4.7
Dry baseline range	4.11	Path weight	4.4
Dry, wet and unknown bin	4.7	Quality status	4.11
Dry period	4.7	Quantisation	3.2
Event	4.7	Radar coverage	4.4
Event pair	4.9	R_{mean}, R_{max}	4.6
Flag	4.3	Scaled MAD	4.3
Floor level	4.3	Significant and heavy event	4.10
Frozen table	4.8	Stage, stage hash	2.2, 2.5
Hop	4.8	Total loss (TL)	3.2
Input signature	2.5	Wet-area fraction	4.7
Intensity class	4.7	Wet core, wet run	4.10
Interval-start labelling	3.6		